

UNIVERSITY OF AMSTERDAM

MASTERS THESIS

Agent-Based Coordination for Military Operations: A Dynamic Task Allocation Framework Under Existing Doctrine

Author:

Koen WEVERINK

Examiner:

dr. Rick Quax

Supervisor:

Artiom van den Broek

Assessor:

David van Tricht

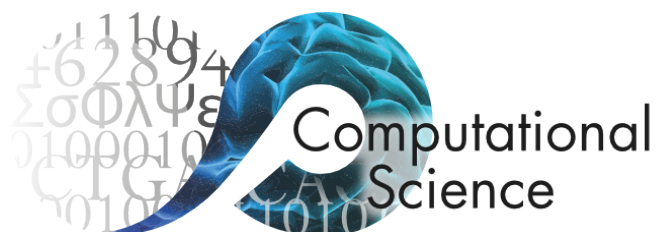
*A thesis submitted in partial fulfilment of the requirements
for the degree of Master of Science in Computational Science*

in the

Computational Science Lab

Informatics Institute

October 2025



Declaration of Authorship

I, Koen WEVERINK, declare that this thesis, entitled ‘Agent-Based Coordination for Military Operations: A Dynamic Task Allocation Framework Under Existing Doctrine’ and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at the University of Amsterdam.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Your signature

Date: 1 August 2020

“The Wheel of Time turns, and Ages come and pass, leaving memories that become legend. Legend fades to myth, and even myth is long forgotten when the Age that gave it birth comes again. In one Age, called the Third Age by some, an Age yet to come, an Age long past, a wind rose in the Mountains of Mist. The wind was not the beginning. There are neither beginnings nor endings to the turning of the Wheel of Time. But it was a beginning.”

-Robert Jordan

Abstract

Faculty of Science
Informatics Institute

Master of Science in Computational Science

Agent-Based Coordination for Military Operations: A Dynamic Task Allocation Framework Under Existing Doctrine

by Koen WEVERINK

This thesis developed and evaluated a framework for dynamic task allocation in military operations, integrating agent-based modeling (ABM), hierarchical task networks (HTNs), and genetic algorithms (GAs) within a two-dimensional grid simulation. The objective was to combine doctrinal interpretability with adaptability under battlefield uncertainty, addressing the need for dynamic planning, integration of heterogeneous unit capabilities, and the generation of plans that remain actionable and understandable across command levels.



Experiments showed that the integration of ABM, HTNs, and GAs was effective for generating doctrinally consistent yet adaptive plans. ABM enabled explicit modeling of heterogeneous units and stochastic combat dynamics, HTNs enforced hierarchical doctrine structures interpretable across command levels, and GAs provided an optimization search method able to find plans that are feasible, acceptable, complete and applicable. Hyperparameter optimization (HPO) further improved performance, with low- and high-fidelity trials sufficient to identify strong parameter settings. Verification runs confirmed robustness to stochastic variation, and transfer tests indicated that the same hyperparameters could guide optimization across different enemy rosters.

Limitations included the computational expense of high-fidelity HPO, the restricted doctrinal coverage in HTN templates, and dependence on the training roster. Because GA and HPO adapted to a fixed set of adversaries, solutions tended to exploit roster-specific patterns rather than general principles. This implies that performance degrades as evaluation rosters diverge from those used in the tuning roster, indicating the need for broader training sets or joint HPO across multiple scenarios.



Keywords: agent-based modeling, genetic algorithms, hierarchical task networks, hyperparameter optimization, military planning, dynamic task allocation

Acknowledgements

Thank the people that have helped, supervisors family etc.

Contents

Declaration of Authorship	i
Abstract	iii
Acknowledgements	iv
Contents	v
List of Figures	viii
List of Tables	ix
List of Algorithms	x
Abbreviations	xi
1 Introduction	1
1.1 Motivation and Context	1
1.2 Opportunities and Challenges of ABM, HTN, and GA	1
1.3 Research Objectives and Questions	3
1.4 Thesis Roadmap	3
2 Literature review	5
2.1 Dutch Military Tactics	5
2.1.1 The Decision Making process	5
2.1.2 Command	8
2.2 Conceptual Models of Planning	10
2.3 Agent-Based Modeling	11
2.3.1 ABM in the Military Domain	12
2.4 Automated Planning	13
2.4.1 Foundations of Classical Planning	13
2.4.2 Hierarchical Task Networks	14
2.4.3 Searching for Plans	16
2.4.4 Genetic Search Algorithms	16

3	Methods	18
3.1	Initializing the Environment	18
3.1.1	River Generation	18
3.1.2	Forest Generation	19
3.2	Utility Functions	20
3.2.1	Vision	20
3.2.2	Pathfinding	21
3.2.3	Tactical Movement	22
3.2.4	Retreating	24
3.2.5	Defend Positions	24
3.2.6	Attacking	25
3.3	Drone	27
3.4	The HTN Planner & Domains	27
3.4.1	Friendly Domain	28
3.4.2	Enemy Domain	30
3.5	Friendly and Enemy Units	32
3.5.1	Friendly Units	32
3.5.2	Enemy Units	33
3.6	The Simulation	34
3.6.1	Behavior Scheduling	35
3.6.2	Plan Evaluation	35
3.7	Genetic Optimization	36
3.7.1	The Search Space	36
3.7.2	GA Environment Initialization	36
3.7.3	Evaluation	37
3.7.4	Crossover and Mutation	37
3.7.5	Selection	39
3.7.6	Generational Optimization Loop	39
3.8	Hyperparameter Optimization	40
3.8.1	Parameter Sampling	40
3.8.2	Evaluation	40
3.8.3	Main Routine	42
4	Experiments and results	43
4.1	Effect of HPO on Performance	43
4.1.1	Fitness Scaling and Normalization	44
4.1.2	Low-Fidelity Trials	45
4.1.3	High-Fidelity Trials	46
4.1.4	Final Picks and Comparison	47
4.2	Verification of Selected Plan	47
4.2.1	No Time Penalty	48
4.2.2	With Time Penalty	49
4.2.3	Comparison	50
4.3	Transfer and Robustness Analysis	51
4.3.1	Roster A: Armor Push with AT Overwatch	51
4.3.2	Roster B: Indirect Fire and Infantry Pressure	51
4.3.3	Comparison Across Rosters	52

5	Discussion	54
5.1	Assumptions and Data Sources	54
5.2	Model Scope and Abstractions	55
5.3	Integration of HTN and GA	55
5.4	Alternative Optimization Methods	56
5.5	Hyperparameter Optimization	57
6	Conclusion	58
7	Ethics and Data Management	62
A	State Templates	64
B	Child Mutation Operations	71
	Bibliography	73

List of Figures

2.1	A conceptual model for dynamic planning [1].	10
2.2	A decomposition tree for the HTN task “move-stack(pla,q)” [1].	15
3.1	Initialization of the grid, including a river, bridges and procedurally generated forests(edges).	19
3.2	Tactical Movement: Staging	23
3.3	Tactical Movement: Flanking	24
3.4	Flowchart illustrating the rule hierarchy of the Friendly Domain. Diamond-shaped nodes represent preconditions; blue ovals denote primitive tasks.	29
3.5	Flowchart of the DelayMission in the Enemy Domain. Decision nodes denote preconditions; blue ovals denote primitive tasks.	31
3.6	Illustration of crossover: (a) parent sequences are partially inherited and (b) recombined into a new child plan.	38
4.1	Gantt chart of the best plan from the verification runs without time penalty.	48
4.2	Gantt chart of the best plan from the verification runs with time penalty.	49
B.1	Mutation operator (a): Shifting a block to different location in the sequence.	71
B.2	Mutation operator (b): Flipping of attack type.	72
B.3	Mutation operator (c): Adding or removing a friendly unit.	72

List of Tables

2.1	Combined overview of weather and terrain aspects, including their military relevance.	7
3.1	Key parameters for tactical movement, threat avoidance, and defensive positioning.	26
3.2	Drone parameters	28
3.3	Parameters used for adjusted score and cost-based evaluation.	41
4.1	Arguments passed to the HPO driver and their roles.	44
4.2	Summary of low-fidelity trials without time penalty.	45
4.3	Summary of low-fidelity trials with time penalty (0.5).	46
4.4	High-fidelity confirmation trials without time penalty.	46
4.5	High-fidelity confirmation with time penalty.	46
4.6	Comparison of final configurations with and without time penalty.	47
4.7	Seed dispersion results for the no-penalty plan.	49
4.8	Seed dispersion results for the time-penalized plan.	50
4.9	Normalized final scalar fitness across rosters and regimes.	53

List of Algorithms

Abbreviations

ABM	Agent- B ased M odeling
ACCA	A nalysis- C onclusion- C onsequences- A ctions (Dutch model)
ACO	A nt C olony O ptimization
AI	A rtificial I ntelligence
CB	C onfirmation B rief
CBRN	C hemical, B iological, R adiological, N uclear
 COA	C ourse of A ction
CSL	C omputational S cience L ab
DOE	D esign of E xperiments
EINStein	E nhanced I SAAC N eural S imulation T ool
FCBB	F inal C ommander's B ack B rief
GA	G enetic A lgorithm
HPO	H yper P arameter O ptimization
HTN	H ierarchical T ask N etwork
ICBB	I nitial C ommander's B ack B rief
ISAAC	I rreducible S emi- A utonomous A daptive C ombat (model)
IWARS	I nfantry W ARfare S imulation
JCATS	J oint C onflict and T actical S imulation
JTLS	J oint T heater L evel S imulation
LVC	L ive- V irtual- C onstructive
M&S	M odeling & S imulation
NSGA-II	N on- D ominated S orting G enetic A lgorithm II
OATDOEM 	O rientation, A nalysis, T errain & W eather, T hreats, M iscellaneous F actors, O wn M eans, and P ossible M odes of A ction

PSO	P article S warm O ptimization
RL	R einforcement L earning
 ROE	R ules of E ngagement
RPM	R ounds P er M inute
RTS	R ea L - T ime S trategy (game genre)
SA	S ituational A wareness
TIM	T oxic I ndustrial M aterials
UvA	U niversiteit v an A msterdam

Chapter 1

Introduction

1.1 Motivation and Context

Military planning provides the formal structure through which operations are conceived, organized, and executed. Rooted in doctrine, it specifies how commanders define objectives, allocate resources, and synchronize the actions of subordinate units across multiple domains. Unlike abstract planning frameworks in computer science, military planning is inherently hierarchical: strategic guidance flows downward through operational and tactical levels, with each echelon translating higher-level intent into actionable tasks [2].

At the same time, military operations unfold in environments characterized by uncertainty. Plans must remain doctrinally consistent while also being sufficiently flexible to adapt when conditions diverge from initial assumptions. Traditional planning methods, however, often struggle to reconcile these requirements. Doctrine provides structured guidance, but the battlefield presents nonlinear interactions, stochastic events, and adaptive adversaries that resist static formulations. As a result, military planning increasingly incorporates computational tools to support decision-making [3]. Such tools enable the exploration of alternative courses of action, the testing of contingencies, and the rehearsal of coordination under diverse scenarios.

1.2 Opportunities and Challenges of ABM, HTN, and GA

Modeling and simulation (M&S) are well-established methodologies in computational science, providing a way to analyze complex systems through digital representations [4]. A system, as described by Zeigler in *Theory of Modeling and Simulation*, may be regarded as a source of data defined by entities, their attributes, the relationships among

them, and the activities that cause change over time [5]. At any given moment, the state of the system is defined by these attributes, while simulation traces how the state evolves under specified conditions.

Computational modeling encodes such elements into algorithms that can be executed on computers. This enables the study of scenarios that would be impractical, unsafe, or too expensive to test in reality. Simulation can thus act as an experimental framework, allowing for the iterative testing of hypotheses, validation of models against empirical data, and projection of system behavior under varying assumptions. Within defense and security contexts, M&S is frequently used as a controlled environment for testing operational concepts that cannot be explored safely or cost-effectively in the field.

Within this broader context, agent-based modeling (ABM) offers one possible approach. ABM represents a system as a collection of agents, each with attributes, internal states, and behavioral rules. These agents perceive their environment, decide on actions based on their states and objectives, and act within the system, typically through a time-stepped or event-driven cycle of sensing, decision, and action [6]. Through repeated execution, ABMs capture how local decisions by individual agents can generate global outcomes. Unlike equation-based models, ABM can represent heterogeneity, adaptation, and spatial dynamics explicitly [7].

The main disadvantage of ABM is its complexity. These models require significant effort to develop, verify, and validate; they also demand more computational resources, and their outputs can be harder to interpret. Nonetheless, in domains where heterogeneity, adaptation, and spatial dependence are central, ABM provides the necessary level of detail, even at the cost of added complexity [8].

Hierarchical Task Networks (HTNs) can be used to structure the planning process by embedding doctrinal knowledge directly into the search space. HTNs ensure that generated plans remain consistent with established military practice, since actions are decomposed along doctrinally prescribed lines [1]. At the same time, HTNs can be rigid if the set of encoded templates is too narrow, and they require careful knowledge engineering to remain useful across different operational contexts.

For the optimization process itself, Genetic Algorithms (GAs) are a suitable candidate. GAs belong to the class of population-based metaheuristics, which preserve multiple candidate solutions simultaneously and are less sensitive to noisy fitness evaluations. They have been applied in domains such as scheduling, routing, and adversarial games [9, 10]. GAs can maintain diversity across a population of solutions, thereby balancing

exploration of new strategies with exploitation of known good ones. Their main disadvantages are computational cost and sensitivity to representation design, which require careful consideration when applied to complex problems.

Taken together, ABM, HTNs, and GAs provide complementary strengths. ABM offers the resolution required to model heterogeneous forces under uncertainty, HTNs constrain solutions to doctrinally meaningful structures, and GAs provide a mechanism for exploring and optimizing plans under stochastic conditions. Each also introduces specific limitations, and addressing these limitations is necessary for their integration into a coherent planning framework.

1.3 Research Objectives and Questions

The objective of this research is to develop an agent-based task allocation framework that embeds doctrinal principles into the coordination of heterogeneous forces in dynamic environments [9, 11]. Implemented in a simplified two-dimensional grid simulation, the framework seeks to produce mission plans that are robust, adaptive, and doctrinally aligned, meeting established military criteria of feasibility, acceptability, completeness, distinctness, and applicability [2].

From this aim arise the central research questions:

- 1. Dynamic Replanning in Hierarchical Frameworks:**

How can classical planning be extended to support dynamic replanning in response to unpredictable combat conditions?

- 2. Integration of Constraints and Heterogeneous Capabilities:**

How can an agent-based planning framework effectively integrate logistical constraints, terrain considerations, and the diverse capabilities of heterogeneous units while ensuring that the generated plans remain interpretable and aligned with established military doctrine?

- 3. Human-Machine Collaboration and Usability:**

How can the planning system be designed to support effective human-machine collaboration, ensuring that the automated plans are not only viable in simulation but also interpretable and actionable by military decision-makers?

1.4 Thesis Roadmap

This thesis is organised as follows:

- **Chapter 2: Literature Review** – Surveys existing approaches to military planning, agent-based modeling, automated planning and optimization, showing their strengths and limitations.
- **Chapter 3: Methods** – Presents the details of the integration of hierarchical task networks, genetic algorithms, and the agent-based simulation environment used in this work.
- **Chapter 4: Experiments and Results** – Describes the experimental setup and reports the outcomes of hyperparameter optimization, verification runs, and robustness tests.
- **Chapter 5: Discussion** – Interprets the findings in relation to the research questions, addresses assumptions and limitations, and outlines possible implications.
- **Chapter 6: Conclusion and Future Work** – Summarises the main contributions and identifies directions for further research.

Chapter 2

Literature review

2.1 Dutch Military Tactics

This section describes the command structure of the Dutch Royal Army as outlined in IK 2-17, 10th impression [2]. Central to this structure is the decision-making process, which unfolds in three distinct phases. The first is the conceptualization phase, guided by the Dutch OATDOEM model (Orientation, Analysis, Terrain and Weather, Threats, Miscellaneous Factors, Own Means, and Possible Modes of Action). This model ensures that planning begins with a comprehensive assessment of both the environment and the available capabilities. The second phase is judgment formation, during which the conclusions drawn from the conceptualization process are synthesized to identify and evaluate potential courses of action (COA). The final phase is the decision itself, where commanders select the course judged most appropriate to the mission and context.

Complementing this process is the Dutch command procedure, which emphasizes clarity and structure in the dissemination of orders. This is formalized through the five-paragraph order, a standardized format that ensures all echelons receive consistent guidance regarding the situation, mission, execution, support, and command arrangements. Together, the phased decision-making cycle and the five-paragraph order form the backbone of Dutch tactical command.

2.1.1 The Decision Making process

Conceptualization Phase

Orientation. First, there is an orientation to the mission. The unit may receive a warning order from higher command or learn of changed circumstances. Based on this information, the unit develops its own warning order which includes creating or

adjusting a schedule (e.g., timings and activities), analyzing the situation for any changes to tasks, and issuing a warning order to the next level of command if necessary.

Next, the unit reviews the higher-level operation order. This involves examining both the mission and the intent of the two next-higher commanders, determining the unit's own role and objectives within the broader context, and identifying any constraints or freedoms in execution. If anything remains unclear, questions are asked to ensure full understanding.

Finally, a Confirmation Brief (CB) is conducted. This is typically an open-form discussion in which participants verify the main problem or threat, restate the new or modified mission, consider how the unit's mission fits into the larger plan (including the rationale behind certain actions), clarify coordination needs with adjacent units (who, when, and how), and address any outstanding issues.

Analysis. Here, the ACCA-model is predominantly used: an **A**nalysis leads to a **C**onclusion, which gets translated into **C**onsequences, that could lead to possible **A**ctions. During the analysis phase, relevant data are evaluated in terms of both time and space, linking them to the adversary's capabilities and/or intentions (e.g., combat, combat support, combat logistics, command and control). Planners also consider the constraints, limiting factors, and freedoms inherent in the assignment, as well as critical points and potential risks. Various methods or templates may be used to structure this analysis, and any resulting conclusions help determine whether the mission remains feasible. If uncertainties persist, questions are raised and clarified.

Following the analysis, an Initial Commander's Back Brief (ICBB) is conducted. This involves restating the mission or objective, confirming the commander's intent, and presenting an outline of the operational concept or plan. Any constraints, limiting factors, and essential coordination requirements are revisited to ensure alignment with higher-level directives. The plan is then refined, with particular attention to coordinating and synchronizing tasks across all involved elements.

Lastly, if necessary, a new warning order is issued at the appropriate level. This warning order may address further preparations such as training, logistics, or command and control. Any remaining questions or issues are addressed before finalizing the plan.

Analysis of TDOEM. Terrain and weather, threats, miscellaneous factors, and own means are assessed. Table 2.1 presents the general aspects of weather and terrain and highlights the factors that can influence military operations. Analyzing these aspects, along with their impact on potential COAs, anticipated threats, and other relevant actors, are then integrated into the subsequent steps of the DOE process.

Factor	Aspect	Military Relevance
Weather	Light	Affects visibility and targeting
	Precipitation	Influences mobility, logistics, morale
	Cloud Cover	Impacts air operations and observation
	Temperature	Shapes endurance, equipment reliability
	Wind	Alters smoke/CBRN, aviation stability
	Visibility	Limits reconnaissance and engagements
	Humidity	Influences soldier endurance, sensors
Terrain	Relief	Determines movement, defensive potential
	Drainage	Affects mobility and supply routes
	Vegetation	Provides cover, concealment, ambush sites
	Soil	Influences construction, vehicle mobility
	Infrastructure	Impacts logistics, maneuver corridors
	Obstacles	Shape maneuver, channel movement
	Approaches	Enable advance or retreat routes
	Key Areas	High-value positions, dominant features
	Observation/FoF	Governs surveillance and firing arcs
	Cover/Conceal.	Essential for survivability and tactics

TABLE 2.1: Combined overview of weather and terrain aspects, including their military relevance

Threats are considered in terms of the six Ws: Who, What, Where, With What, When, and Why. Both the most likely and the most dangerous threat scenarios should be developed, based on the most probable threat scenario established at higher command levels.

Miscellaneous factors include rights, duties, limitations, or rules of engagement (ROE); the involvement of additional active actors (such as international or national governmental and non-governmental organizations); religious or local cultural influences; local laws or regulations; media dynamics; and potential threats from chemical, biological, radiological, nuclear (CBRN) agents or toxic industrial materials (TIM).

Own means must also be evaluated, considering which unit will be involved (Who), what assets are available—including their deployability, capacity, training, and experience (What), when these assets can be brought into action (When), and the extent to which they may be employed, taking into account any command or support relationships (Permitted Use).

Judgment Formation Phase

Based on the sub-conclusions from the preceding steps (ATDOE), the conclusion points to developing possible courses (Modes) of action. These courses must align with the functions of military operations: command and control, information and intelligence,

striking power, protection, and logistics. Plans must also be feasible, acceptable, complete, distinct, and applicable. In addition, they require a clear time or activity schedule and must reflect orders from both the next higher commander and one's own commander.

Once these possible COAs have been established, they are compared by assessing the opportunities and risks inherent in each, evaluating them against success criteria, and selecting the most suitable option according to self-defined selection criteria.

Decision Phase

First, tasks or assignments are allocated to specific units, and any necessary reserves and escalation measures are identified. The plan is then developed, including an operational concept, a clear center of gravity, and designated reserve forces. Additionally, a support plan and a communication or media plan are prepared to ensure adequate backing for the operation.

Next, if feasible, a reconnaissance is conducted based on a reconnaissance plan. The findings provide a preliminary impression of the terrain and help determine whether the concept plan remains realistic.

Then, the Final Commander's Backbrief (FCBB) takes place, where the plan is presented orally in the commander's own words, supported by a (concept) operational scheme or sketch. During this briefing, the overall concept and its supporting plans are evaluated, along with a review of potential scenarios, associated risks, and any mitigation measures. Higher command may offer comments or modifications, after which a final decision is made.

Lastly, if necessary, higher-level command conducts a rehearsal. Ultimately, a final decision is reached, and the operation order is issued.



2.1.2 Command

Orders are preferably given verbally in the field or using a command-support system, a sketch, or a scale model. If possible, a reconnaissance of the terrain should be conducted beforehand. Once the unit has assembled, the commander provides a terrain and/or map orientation. The orders themselves follow the standard five-paragraph format, as outlined below.

Situation. The situation begins with an overview of the threat and other relevant factors: who constitutes the threat or factor, what actions they may take, where and when they might take them, why they might act, and how they could carry out these actions.

Next, the status of friendly troops is explained, including subordinate or adjacent operational concepts and any responsibilities for mutual support. The commander also defines subordinate command relationships, detailing how units will move and operate within their assigned areas, along with the relevant command relationships and any additional instructions or signals (for instance, alert methods or transport procedures). Finally, the commander provides an evaluation of the situation, which may vary depending on the circumstances.

Assignment and Execution. First, the received assignment is briefly and concisely displayed. Then, the plan addresses three core elements in one coherent narrative:

- Purpose (goal): “What role does my unit play in the higher-level plan?”
- Method (approach): “How should my unit carry out the mission?” (including key operational decisions)
- End State (intended final situation or effect): “Which actual effect or end state must be achieved?”

The operational concept is a verbal overview of the plan, potentially broken down by phases. It includes coordination with any supporting units or resources and outlines how the adversary will be addressed. The plan should consider the unit’s capabilities and the influencing circumstances. “What if” scenarios are formulated to account for possible contingencies.

Tasks are then assigned to combat units, followed by tasks for combat logistics and command-support units. Instructions are also given for the reserve, clarifying how and when it may be employed.

Finally, coordinating provisions apply to all subordinate units. These may include timings, rendezvous points for coordinating with other commanders, ROE, recognition signals, communications protocols, transport arrangements, vehicle usage, and media considerations.

Logistics. The logistics plan, which may be divided into phases, describes the intended effects and specifies any obligations or constraints. It encompasses both materiel and personnel logistics. Materiel logistics addresses supply, maintenance, movements, logistical services, and infrastructural support. Personnel logistics focuses on personnel management, welfare, medical care and casualty evacuation. Additionally, the plan identifies logistical areas, routes, and priorities, along with any subordinate logistics commands or support relationships (including their duration).

Command and Control. This section specifies the command posts of higher-level units, as well as the unit's own command post—both current and planned. It also establishes how command will be transferred if needed. A communications plan details any special code words that differ from the standard operating procedures, outlines restrictions on the use of electronic communications, and describes contingency measures to be taken if communications fail.



2.2 Conceptual Models of Planning

One of the most influential abstractions in computational planning is the conceptual model introduced by Ghallab, Nau, and Traverso in *Automated Planning: Theory and Practice* [1]. In this framework, planning is framed as the interaction of three core components: the state-transition system, the planner, and the controller (See Fig 2.1). The state-transition system evolves according to a transition function driven by actions and events. The planner constructs a plan by taking as input a description of the system, an initial state, and a goal condition, producing a sequence of actions intended to achieve that goal. The controller then executes the plan by applying these actions to the system, observing state changes, and continuing until the objective is achieved or the plan fails.

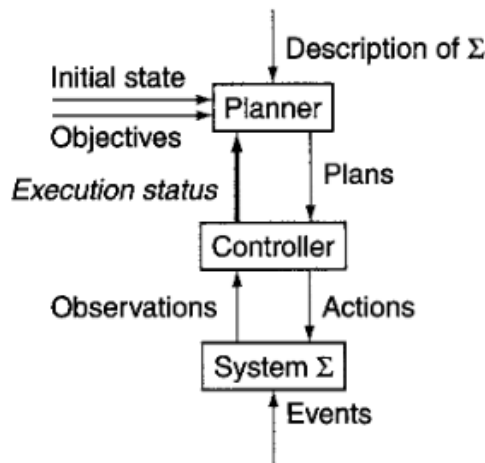


FIGURE 2.1: A conceptual model for dynamic planning [1].

However, the model rests on several assumptions that limit its applicability in domains such as military operations [12]. It assumes that the state space is finite, yet the combination of continuous time, granular movement, and diverse actions make the effective state space unbounded in combat. It presumes deterministic action outcomes, but the stochastic nature of weapon fire and battlefield uncertainty defies this assumption. Furthermore, it assumes sequential ordering of actions, whereas military units often act simultaneously, and coordination is essential to operational realism. The model further

reduces goals to static predicates, though combat objectives are often multifaceted and shifting in response to evolving conditions. It presumes that planning is conducted offline before execution, while in practice plans must be continually revised to remain viable. Observability is also assumed to be complete, though real-world intelligence is partial and uncertain. Finally, the adversary is treated as static, when in reality opponents are adaptive agents capable of strategic and unpredictable behavior.



These divergences show both the strength and the weakness of the classical conceptual model. For structured, deterministic environments, it offers an effective method. In dynamic, adversarial, and uncertain settings such as combat, however, the assumptions of the model collapse, producing unrealistic or infeasible plans. This has motivated research into alternative approaches that preserve computational power while relaxing unrealistic constraints. Hierarchical task networks, probabilistic planning methods, and agent-based frameworks are ways to achieve this. Among these, agent-based modeling is particularly well-suited for military contexts, as it enables the representation of heterogeneous and adaptive units whose interactions potentially generate emergent operational outcomes. The present thesis aims to embed military doctrinal knowledge into an agent-based task allocation framework, bridging the gap between abstract planning models and the realities of military operations.

2.3 Agent-Based Modeling

Agent-based modeling (ABM) is an M&S method in which systems are represented as collections of discrete, autonomous entities whose interactions can lead to emergent system-level behavior. Each agent possesses a set of attributes (state variables), a repertoire of possible actions, and interaction rules governing how it perceives and affects its environment [6]. The overall system state at a given time is defined by the configuration of all agents and their relationships, and simulation proceeds by observing how this state evolves as agents execute actions and respond to change. This bottom-up representation allows ABM to capture forms of complexity that cannot be expressed in closed-form equations.

ABM simulations typically unfold through a repeated sense–decide–act cycle, often implemented in discrete timesteps or as event-driven processes [13]. Agents perceive their surroundings, update their internal states, and act accordingly. Through many iterations and local interactions they generate nonlinear and adaptive collective behavior. Much of the modeling effort lies in defining the behavioral rules, since small local adjustments can propagate into substantial global effects. While this expressiveness enables ABM to model heterogeneity, adaptation, and spatial dependence explicitly, it also increases

verification difficulty and computational demand. Consequently, ABM is most appropriate when individual variability and interaction structure are central to the system being modeled.

2.3.1 ABM in the Military Domain

The military domain exemplifies conditions where ABM is not merely advantageous but necessary. Combat involves heterogeneous, adaptive forces whose coordination and decision-making cannot be reduced to aggregate equations [7, 13]. Defense organizations therefore employ the Live–Virtual–Constructive (LVC) paradigm [14], where live exercises combine with synthetic environments for analysis and training. Large-scale constructive systems such as the Joint Theater Level Simulation (JTLS) and the Joint Conflict and Tactical Simulation (JCATS) integrate air, land, and naval operations into campaign-level simulations [3, 15]. However, their reliance on aggregated or scripted behavior limits the representation of adaptive tactics.

Agent-based combat models address this gap by explicitly representing combatants and units as autonomous entities whose local decisions generate operational outcomes. Early examples include Ilachinski’s *Irreducible Semi-Autonomous Adaptive Combat* (ISAAC) and its successor, the *Enhanced ISAAC Neural Simulation Tool* (EINStein), which conceptualized warfare as a complex adaptive system populated by perceptive, mobile, and social agents [16]. Later frameworks such as the Infantry Warrior Simulation (IWARS) and CombatXXI expanded fidelity by incorporating diverse weapons systems, environmental effects, and communication degradation [17, 18]. These models demonstrated how decentralized decision rules could reproduce maneuver, cohesion, and emergent combat dynamics.

Together, these studies show both the power and the difficulty of ABM in defense research. Its fidelity and realism enable exploration of adaptation and coordination under uncertainty, yet the resulting complexity complicates validation and interpretation. Building on this tradition, the present work adopts ABM as the foundation for a task-allocation and coordination framework in which agents operate under doctrinal constraints rather than purely emergent rules. By embedding formal doctrine within agent behavior, the model aims to retain the adaptive, nonlinear character of combat while ensuring that generated plans remain interpretable and consistent with established tactical logic.

2.4 Automated Planning

2.4.1 Foundations of Classical Planning

Automated planning concerns the generation of action sequences that modify a modeled environment so that a specified goal state is achieved [1]. A planning system uses a plan generator that applies a set of formal rules to a planning problem, producing an executable plan designed to achieve those goals. The human user defines the planning problem, which, along with the current configuration of the world model, is translated into a set of parameters that serve as the input to the planner. Since the world is represented by an abstract model of reality, the accuracy and reliability of the generated plan depend on the model's fidelity. Static models enable the construction of plans with predictable outcomes, while stochastic models introduce uncertainty that complicates goal satisfaction.

In classical planning, the world is modeled as a collection of states, each represented by a set of atomic logical propositions that are either true or false. Actions modify the state by changing these truth values. The planner searches for a sequence of actions that transitions the system from an initial state s_0 to a goal state that satisfies desired conditions. The environment is assumed to be deterministic, fully observable, and static, meaning that only planner-generated actions can alter the state.

Formally, the state transition is defined through the function γ , which maps the current state s and an action a to a new state s' . Actions are instantiated from operators that specify variables, preconditions, and effects. An operator is defined by the tuple

$$(v, p^+, p^-, e^+, e^-),$$

where v denotes variables, p^+ and p^- are positive and negative preconditions, and e^+ and e^- are positive and negative effects, respectively. By applying a substitution σ that maps variables to constants, we obtain a concrete action a . This action is applicable if all its positive preconditions are present in the current state s and none of its negative preconditions appear in s . The resulting state-transition function is given by

$$s' = \gamma(s, a) = s - \sigma(e^-) \cup \sigma(e^+). \quad (2.1)$$

A plan is therefore a sequence of actions leading from the initial state s_0 to a goal state g . A goal is typically defined by desired conditions g^+ and forbidden conditions g^- , so a state s satisfies the goal if all g^+ hold and none of g^- do. A plan is valid if each action's

preconditions hold in the state where it is applied. For example, a_1 is valid in $\gamma(s_0, a_0)$ and a_2 in $\gamma(s_1, a_1)$, and so on.

Classical planners generally use one of two search methods. *Forward planning* applies actions beginning from s_0 , iteratively generating successor states until a goal is reached, backtracking as necessary. *Backward planning* (or regression planning) begins from the goal conditions and applies inverse operators, regressing over actions, until a state consistent with s_0 is found. However, these methods are limited by the exponential growth of the search space and by assumptions of determinism and full observability that often do not hold in dynamic, adversarial environments such as military operations.

Despite these limitations, classical planning provides the theoretical foundation for more advanced frameworks. It establishes the formal basis for hierarchical, probabilistic, and learning-based approaches that relax or extend its assumptions to address uncertainty and complexity. The next section builds on this foundation by introducing Hierarchical Task Network (HTN) planning, which incorporates structured domain knowledge to constrain and guide plan generation.

2.4.2 Hierarchical Task Networks

Hierarchical Task Network (HTN) planning extends the classical planning by introducing structured task decomposition [19]. Whereas classical planners search directly over primitive actions, HTN planners operate over a hierarchy of abstract tasks that are recursively refined into simpler subtasks until executable actions are reached. This approach embeds domain knowledge into the planning process, allowing planners to exploit meaningful “chunks” of behavior rather than reasoning at the atomic action level.

The origins of hierarchical planning trace back to STRIPS [20] and its extension through macro operators (MACROP) [21], which enabled recurring sequences of primitive actions to be combined into higher-level operators. HTN planning generalizes this idea by providing a formal mechanism for multi-level decomposition guided by domain-specific methods. This structure reduces the search space, improves computational efficiency, and produces plans that are easier to interpret and repair.

An HTN planning problem is defined by an initial task network and a domain that specifies decomposition methods and action preconditions. A solution is a fully refined task network in which all tasks are executable primitives. Typically, planning begins with a single top-level task—often labeled *Act* [9]—which is recursively expanded until a valid

sequence of primitive actions is obtained. Standard search techniques such as depth-first, graph, or iterative-deepening search are applied to explore possible decompositions [1]. The resulting plan can be visualized as a decomposition tree, where each non-primitive task is replaced by its subtasks until terminal leaves represent concrete actions. Figure 2.2 illustrates such a tree for the task “move-stack(pla,q)” [1].

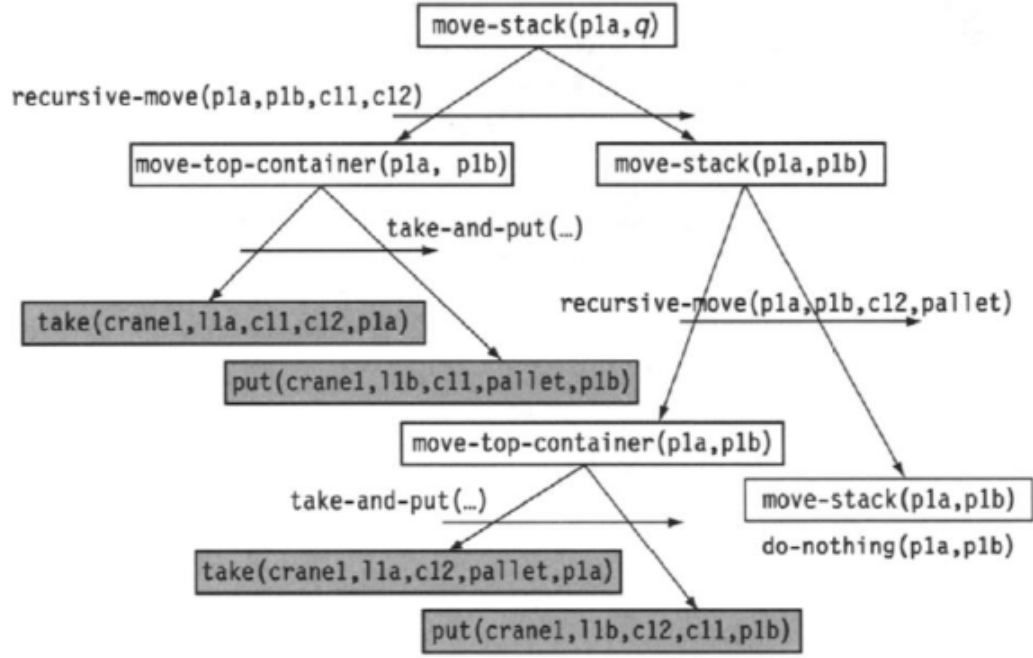


FIGURE 2.2: A decomposition tree for the HTN task “move-stack(pla,q)” [1].

HTN planning’s strength lies in its explicit representation of procedural knowledge. The Domain encapsulates both valid task sequences and their preconditions, enabling planners to construct logically sound, doctrinally consistent plans. Although the need to define decomposition methods constrains flexibility, it substantially improves efficiency and expressiveness, allowing HTN planners to model problems that classical methods cannot [1].

Beyond robotics and logistics, HTN planning has been widely adopted in game AI. Early implementations coordinated small teams and non-player characters in first-person shooters [22–24], while more recent systems apply HTN-based reasoning to multi-agent coordination in real-time strategy games. Examples include Muñoz-Avila and Aha’s limited HTN planner for Stratagus [25], Ontañón et al.’s case-based method extraction from human demonstrations in Wargus [26], and Ontañón and Buro’s integration of HTN planning with minimax search in the μ RTS environment [27]. These applications demonstrate HTN’s ability to combine structured knowledge representation with adaptive control in dynamic domains.



2.4.3 Searching for Plans

According to Russell and Norvig (2010), effective search comprises five components: search nodes, a successor function, a cost function, a goal test, and a search strategy [9]. In this thesis, these elements will be defined within the framework of HTN planning, which seems to be well-suited for addressing the problem at hand.

In HTN planning, the *search node* is typically the task network together with any associated constraints and variable bindings. The *successor function* takes this network and produces new ones by applying methods to decompose nonprimitive tasks into subtasks. A *cost function* is not always required, since many HTN planners treat the problem as one of finding any valid plan [19]. When costs are considered, simple measures such as the total number of primitive actions in the final plan may be used, or, if actions have different weights, the sum of their individual costs. In temporal domains, elapsed time may serve as the cost, and in applied settings such as combat modeling, more complex criteria like expected casualties or probability of success can also be incorporated [12]. Finally, the *goal test* checks whether every nonprimitive task has been fully decomposed into executable operator instances and all constraints are satisfied.

The *search strategy* is the most complex element. It is the part of the search algorithm that chooses which successor node to expand next. When minimizing a cost function is essential, the chosen search strategy must rapidly identify plans with the lowest possible cost. If the cost consistently increases along every branch of the planning process, uniform cost search will reliably discover the optimal plan before encountering any sub-optimal ones. Moreover, when a heuristic is available alongside this cost function, A* search can have increased performance over uniform cost search. In situations where obtaining a solution is critical regardless of cost trends, especially when the cost does not steadily increase with each planning step, a breadth-first search might be necessary. Conversely, if cost is irrelevant and multiple acceptable solutions exist, depth-first search may offer a faster, more memory-efficient method, even if the solution is not optimal. For cases where speed is prioritized over guaranteed success (accepting that the algorithm might sometimes fail to find a solution), methods like greedy or beam search can be effective. Ultimately, the effectiveness of any search strategy is closely tied to the specific features of the domain [9].

2.4.4 Genetic Search Algorithms

A genetic algorithm (GA) is a type of search algorithm inspired by evolutionary biology that generates new states by combining two or more parent states rather than by simply

altering a single state [9]. It starts with a *population* of k randomly generated individuals, each encoded in a form suitable for the problem at hand; be it a binary string, a sequence of digits, or another representation. The algorithm proceeds by evaluating each individual with a fitness function that assigns higher values to better solutions. Next, pairs of individuals are randomly selected for reproduction, and for each pair, a crossover point is chosen at random to combine segments of the parents' representations. This produces offspring that may differ significantly from either parent. This means that early in the process, when diversity is high, crossover can result in large exploratory steps. Later, as the population becomes more similar, the changes tend to be smaller. Finally, a random mutation is applied to each position in the offspring with a small, independent probability.

The power of genetic algorithms lies in their ability to merge substantial portions of high-quality solutions from different individuals. However, the success of a GA depends on the careful design of the individual representation. If a problem cannot be effectively encoded into a format like a string of bits or digits, the algorithm is unlikely to produce accurate results.

Genetic algorithms have proven successful in a variety of optimization domains, including circuit layout and job-shop scheduling [9], as well as in real-time strategy (RTS) games [10]. While they show promise for maneuver planning, a limitation of genetic algorithms is their lack of a direct mapping to other tactical considerations in combat modeling.



Chapter 3

Methods

This section shows the development of a planning framework that integrates three complementary components: hierarchical task networks (HTNs) to encode doctrinal structure, a genetic algorithm (GA) to optimize plan variants under stochastic conditions, and a custom agent-based simulation environment to evaluate outcomes. HTNs provide a structured representation of feasible actions, making sure that generated plans remain interpretable and doctrinally consistent. The GA operates over these representations, evolving populations of candidate plans through crossover, mutation, and selection. The simulation environment models heterogeneous units, terrain constraints, and stochastic combat dynamics, enabling fitness evaluations that reflect both operational effectiveness and variability across seeds. Together, these methods form a workflow in which doctrinally grounded plans are generated, optimized, and tested under uncertainty.

3.1 Initializing the Environment

A stylized two-dimensional grid is constructed to represent the environment, as illustrated in image 3.1. Its dimensions are set by default to `grid_width = 100` and `grid_height = 75`, with each cell corresponding to a unit square of `cell_size = 100`. All positions are defined on the integer lattice (x, y) within this bounded rectangle. The enemy unit have already chosen their Defend Positions. Friendly units always start at $(0, 75)$.

3.1.1 River Generation

The river is introduced as a fixed feature extending across the map, as shown in image 3.1. Its course is designed to include three distinct sections, thereby imposing topographical

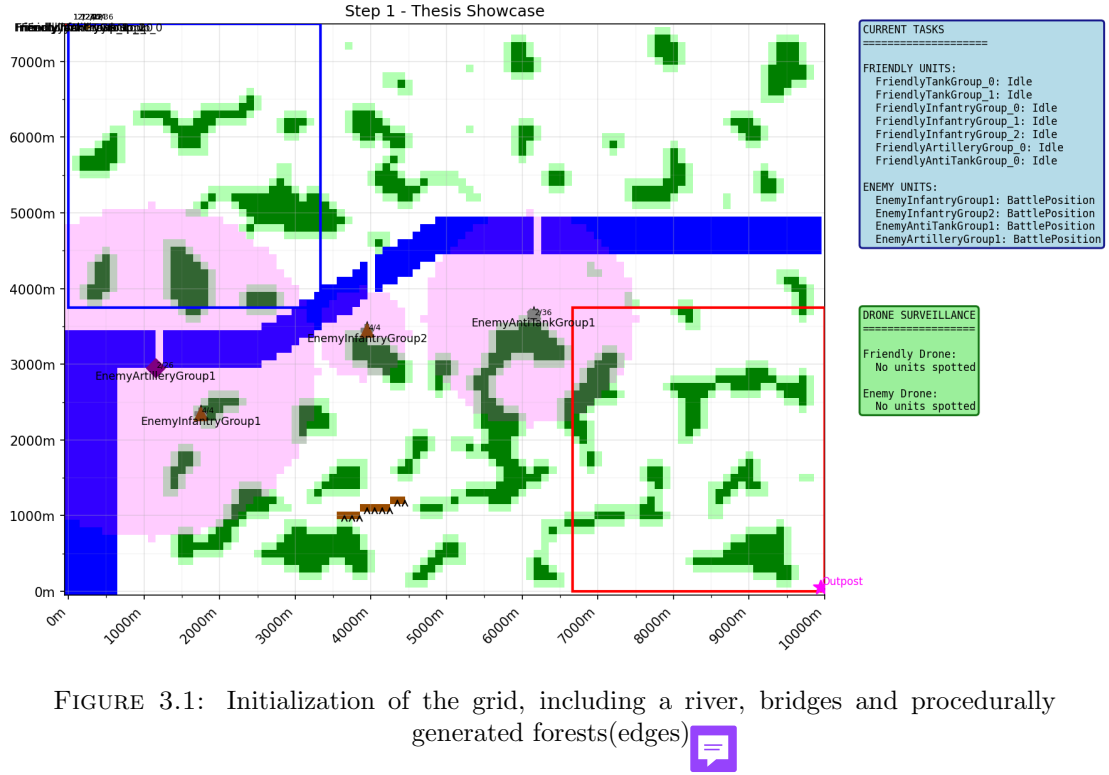


FIGURE 3.1: Initialization of the grid, including a river, bridges and procedurally generated forests(edges)

constraints on the simulation. The first section consists of a seven-column vertical strip aligned with the western edge. This segment is extended to the grid boundary at row zero to ensure continuous contact with the map edge. The central section is then formed as a horizontal band covering columns 7–24 and rows 20–24. A diagonal connector follows, spanning columns 25–49, with five parallel rasterized lines stepping upward from rows 20–24 to rows 35–39. The river terminates in a final horizontal section, a band from column 50 to the eastern boundary at rows 35–39.

To allow controlled passage across the river, three single-tile crossings are cleared to act as bridges. One bridge is positioned at $x = 12$ in the first horizontal section, a second consists of one cleared cell per diagonal line near $x = 40$, and the final bridge is located at $x = 62$ within the last horizontal section. These crossings provide opportunities for interaction between regions on either side of the river.

3.1.2 Forest Generation

Forests are procedurally generated as follows: each cell not occupied by the river could be initialized independently through a Bernoulli trial with success probability $p = 0.45$. This produces the initial stochastic distribution of trees [28]. This value is chosen to approximate a near-critical balance between occupancy and vacancy, thus making sure there is sufficient density to generate coherent clusters while avoiding near-total coverage that would suppress meaningful spatial variation. Random draws are obtained from the

standard pseudorandom number generator¹, with a fixed seed to ensure reproducibility across experiments.

This preliminary distribution of forest tiles is then regularized using a neighborhood-based smoothing procedure. For each cell, the Moore neighborhood (comprising the eight surrounding cells) is examined [29]. A rule is then applied uniformly: an occupied cell persists if it has at least four occupied neighbors, whereas an empty cell becomes occupied if it has at least five. These thresholds are selected to favor the growth of locally dense clusters while suppressing sparsely distributed cells. Four iterations of this update are performed, a number empirically found sufficient to eliminate isolated noise without impeding the overall variability of the forest. The outcome is a set of contiguous forest masses.

A final boundary adjustment is applied to refine the forest. Any cell identified as forest but adjacent to at least one non-forest cell is classified as a forest edge. This edge has different properties from the regular forest, which will become apparent in following sections of the thesis.

3.2 Utility Functions

This module defines the mechanics that govern how units perceive, navigate, and act in the environment. Vision combines line-of-sight with a vision range, while taking into account cover and stealth. Pathfinding employs A* with terrain-based costs. Tactical movement extends these foundations by evaluating candidate positions for staging or flanking based on terrain preference, exposure, and path length. Retreating performs a bounded search to locate covered, out-of-range positions. Defensive positions are selected relative to outposts or bridges using terrain rewards, field-of-fire checks, and crowding penalties. Combat is resolved probabilistically, accounting for rate of fire, suppression-modified accuracy, armor penetration via a piecewise probability map, and health loss. Together, these components form a unified framework for perception, movement, and combat to simulate higher-level behaviors.

3.2.1 Vision

A location is considered visible to an enemy if three conditions are satisfied: the enemy must be alive, the Manhattan distance to the location must not exceed the effective vision range, and the location must fall within its line of sight [30]. The stealth value associated

¹<https://docs.python.org/3/library/random.html>

with the observing unit is drawn from its state variables, which can be referenced in Appendix A. Cover is defined as being in the forest (including the forest edge). For line of sight (LOS), a Bresenham-style line is drawn between the friendly and the enemy unit [31]. LOS is obstructed by forest tiles but not by edge tiles.

Perception integrates line of sight with a range attenuation model, reducing vision when the observed position lies within cover and when the target possesses a stealth modifier. This formulation reflects the assumption that being in cover and stealth act to decrease detection. The effective range R_e is defined as

$$R_e = \frac{R_b}{1 + \frac{S}{cell_size}}, \quad (3.1)$$

where R_b denotes the base range in open terrain and S represents the stealth parameter of the observed unit, with $S \in [0, 100]$. The denominator scales the reduction in range relative to the grid resolution. This formulation was selected because it allows cover and stealth to act on perception without abruptly shortening visibility, while still permitting sufficiently high stealth values to reduce effective range.

For illustration, consider an enemy with a base range of $R_b = 12$ cells observing a target located at a Manhattan distance of 10. If the target has a stealth modifier of $S = 50$ and is positioned within cover, the effective range is reduced to

$$R_e = \frac{12}{1 + \frac{50}{100}} = \frac{12}{1.5} = 8. \quad (3.2)$$

Since the actual distance exceeds this reduced range, the target remains undetected despite being within the nominal base range.

3.2.2 Pathfinding

Pathfinding is performed using the A^* algorithm applied over the set of admissible neighboring cells, with movement restricted by impassable terrain such as rivers. The baseline movement cost is set to one per cell. Unit-specific biases are incorporated to reflect terrain preferences: infantry incur a higher cost when traversing open ground, heavy units are penalized within cover, and anti-tank teams are biased toward forest-edge positions. These higher costs influence route selection by differentiating otherwise equivalent paths. The value of these penalties can be seen in Table 3.1.

Hostile threat zones are incorporated by assigning very high additional costs to cells within the attack range of enemy units. This does not make such positions strictly

forbidden, but renders them strongly unfavorable, ensuring that exposure is avoided unless no cheaper alternative exists. The heuristic employed is the Manhattan distance to the target location.

The computed trajectory is cached and after ten timesteps the path is recomputed. At each reuse, the path is rekeyed to the current position of the unit, allowing longer motions to be executed without the expense of re-planning at every tick. The value of ten was selected empirically as a compromise: it is sufficiently long to take care of computational cost across multiple steps, yet short enough to preserve responsiveness to environmental changes such as enemy movement or altered terrain conditions.

3.2.3 Tactical Movement

Tactical movement toward enemy contact is determined by evaluating candidate positions in the vicinity of the target and assigning scores that reflect terrain, threat exposure, and path costs. Two primary behaviors are distinguished: staging and flanking.

For staging, candidate cells are sampled on a stride-2 grid within a fixed radius. Sampling at stride two reduces the number of evaluations while preserving spatial diversity. Candidates are immediately discarded if they fall outside the map boundaries, lie within the river, coincide with another enemy's position, or—unless the moving unit is infantry or anti-tank—are located inside forest or forest edge tiles. For artillery, candidates must additionally lie within attack range, while other unit types are constrained by a maximum engagement distance set as a design limit to avoid unrealistically long approaches.

Threat penalties are incorporated to steer units away from hazardous locations. A large additional cost is imposed on candidates lying within any enemy's attack range. This penalty is scaled inversely with distance, reflecting the greater danger of close proximity while still allowing distant threats to exert some influence. An inverse function is chosen over linear or quadratic decay because it provides a smoothly diminishing penalty that remains sensitive at intermediate ranges without either overemphasizing very short distances or prematurely discounting longer ones. The penalty is further multiplied by a higher factor when line-of-sight is present, representing the increased lethality of unobstructed engagements. The value of these penalties can be seen in Table 3.1.

Candidate evaluation requires two A* computations: one path from the unit's current position to the candidate, and another from the candidate to the enemy. Artillery units maximize a score that balances three competing objectives: maintaining longer stand-off distances, minimizing exposure, and preserving a short approach to firing positions.

again

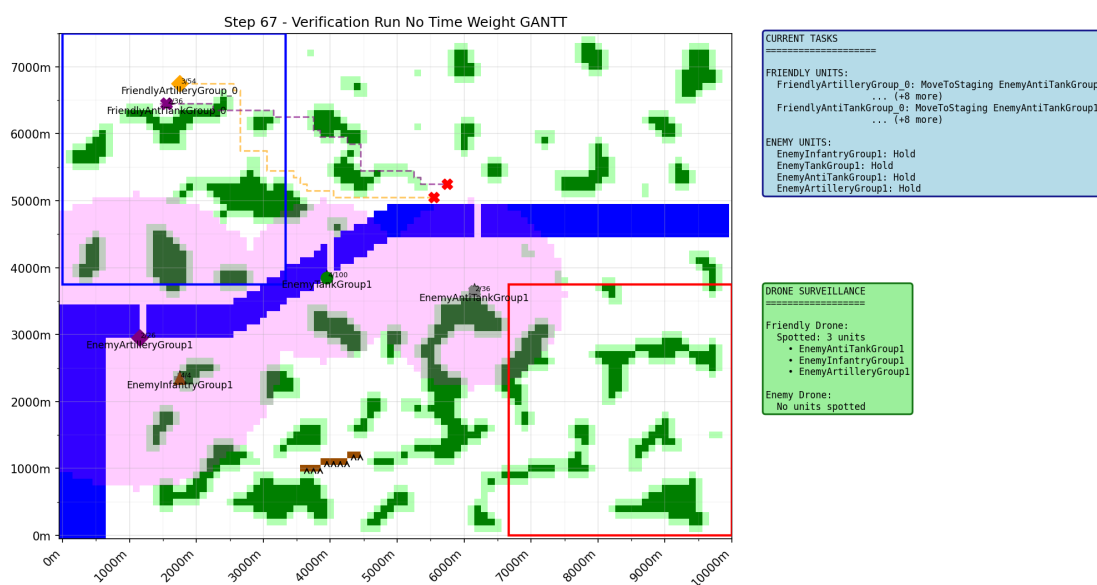


FIGURE 3.2: Tactical Movement: Staging

again

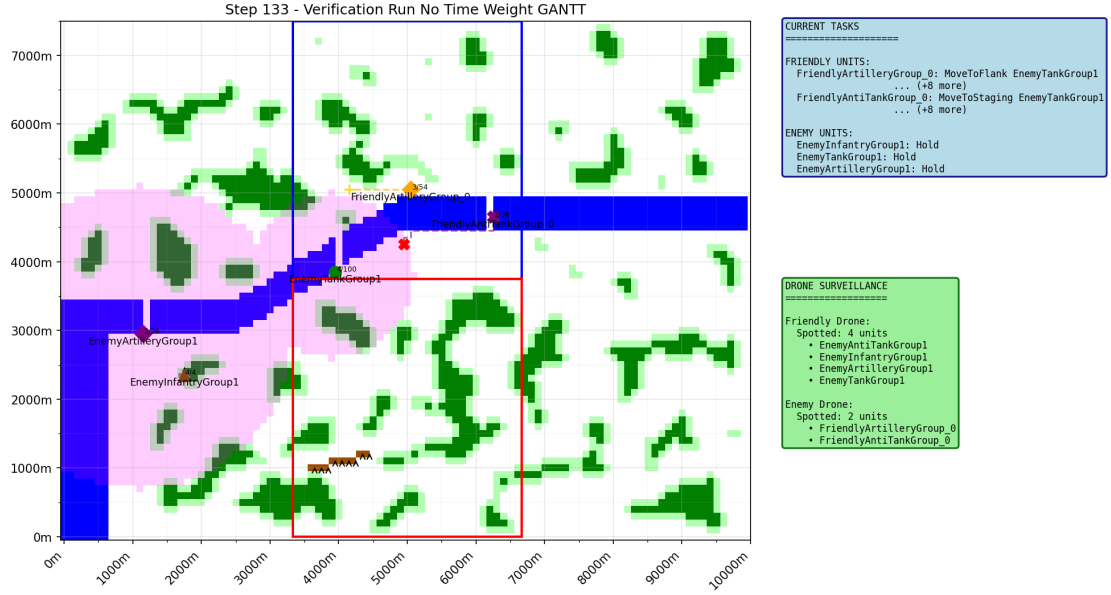


FIGURE 3.3: Tactical Movement: Flanking

3.2.4 Retreating

Retreat positions are selected by evaluating terrain around the nearest opponent within a 21×21 search window centered on that opponent, excluding impassable terrain. Each candidate must be reachable within a maximum A* path length and is scored higher for providing cover, breaking LOS, or lying beyond the opponent's attack range. Exposed positions receive lower scores.

If no satisfactory position is found, a fallback procedure selects the map corner farthest from the opponents' centroid. The A* path toward this corner is scanned until either a position outside LOS and range is reached or the travel budget is exceeded. The first suitable or last reachable point is then returned, ensuring a valid retreat location in all cases.

3.2.5 Defend Positions

Defensive positioning for individual enemies is determined by evaluating candidates in the vicinity of a designated outpost or tactical landmark. At the start of a scenario, the first defend positions are computed near bridges, as these chokepoints represent natural avenues of approach. Candidate locations are sampled within a fixed radius south of the bridge center, excluding impassable terrain such as river cells or dense forest. Each candidate must also be reachable via an A* path from the unit's current position to ensure practical accessibility. Figure 3.1 shows the positions chosen by the enemy units at the start of the simulation.

Scoring incorporates several components. A distance penalty discourages excessive displacement from the bridge axis, thereby concentrating the defense near the objective. Role-specific preferences further refine candidate evaluation: infantry and anti-tank teams are rewarded for occupying forest edges where cover and ambush opportunities are present, whereas other units prefer open ground to preserve visibility and maneuverability. To prevent overcrowding, a crowding penalty is applied when candidates lie close to existing defend positions already assigned to other units. This penalty scales with proximity, discouraging clustering while still allowing limited overlap where tactically beneficial. Equation 3.3 illustrates this scoring mechanism. If no suitable candidate is found, a fallback procedure selects a position immediately south of the bridge, provided it is within bounds and free of impassable terrain. Should this also fail, the unit defaults to retaining its current position.

Once bridge defense has been established, the defend-position routine shifts focus to protecting the assigned outpost. The search is centered on the outpost, using a reference line drawn from the friendly centroid toward the objective as a guiding axis. Candidate positions are evaluated relative to this line: those deviating beyond a set perpendicular distance are discarded, while positions near the boundary incur a minor penalty. Scoring criteria are the same as those in the bridge-defense phase.

The combined scoring function for a candidate position p can be expressed as

$$S(p) = -\alpha d_{\text{bridge}}(p) + \beta R(p) + \gamma F(p) - \delta C(p), \quad (3.3)$$

where $d_{\text{bridge}}(p)$ is the Manhattan distance to the bridge or reference axis, $R(p)$ is the role-specific terrain preference term (positive for desirable terrain, negative otherwise), $F(p)$ is the field-of-fire reward for long-range units, and $C(p)$ is the crowding penalty based on proximity to other defend positions. The coefficients $\alpha, \beta, \gamma, \delta$ weight the relative importance of each factor.

3.2.6 Attacking

Close combat resolution accounts for orientation, firing rate, accuracy, armor, and penetration. Before firing, the attacker's facing is aligned toward the target to ensure that directional effects are applied. The number of attack attempts per tick is generated as a stochastic conversion from the weapon's rounds-per-minute (RPM), with expected value $0.1 \cdot \text{RPM}$. This is done to provide a probabilistic firing cadence that preserves long-term averages while introducing natural variability at short time scales.

The number of successful hits is drawn from a binomial distribution, parameterized by the number of attack attempts and the attacker’s accuracy. Accuracy is reduced proportionally to the unit’s current suppression level, reflecting degraded effectiveness if units themselves are engaged. Each successful hit increases the target’s suppression.

Penetration is resolved independently for each hit. The probability of success is given by a piecewise function (Eq. 3.4) of the difference between weapon penetration and the target’s directional armor rating (front, side, rear). Rear armor is weakest and therefore confers the highest probability of penetration. The function is designed to model realistic armor interactions. It returns (near-)zero probability when penetration is far below armor, a transitional region with gradual increases, and near certainty when penetration substantially exceeds armor.

$$P(D) = \begin{cases} 0.00, & D \leq -3, \\ 0.33 + 0.11(D + 3), & -3 < D \leq 0, \\ 0.66 + \frac{0.29}{6}D, & 0 < D \leq 6, \\ 0.95, & D > 6, \end{cases} \quad (3.4)$$

where $D = \text{Friendly Penetration} - \text{Enemy Armor}$

Damage is proportional to the attacker’s per-shot damage value and the current group size, and cumulative damage reduces group size in integral multiples of a base health constant. A unit is eliminated when either total health or group size reaches zero, at which point the enemy-alive flag is cleared and counters are reset. When a living target is struck, the attacker’s identifier is recorded to enable subsequent reactive behaviors.

Initial values for cumulative damage, suppression, penetration, armor and other related state fields provided by standardized unit templates defined in Appendix A. The principal constants and default thresholds that shape the behavior are collected in Table 3.1 below for reference.

Name / Condition	Value	Role
Open-ground cost (scout/infantry)	+5	Prefer cover
In-cover cost (tank/artillery)	+5	Prefer open
Off-edge cost (anti-tank)	+5	Prefer forest edge
Avoid-zone penalty	+300	Discourage threatened cells
Avoid penalty weights	1000 (LoS), 250 (no LoS)	Threat falloff $1/d$
Long-range field-of-fire test	3 steps cardinal	Reward open arcs
Bridge defense radius	15	Candidate envelope

TABLE 3.1: Key parameters for tactical movement, threat avoidance, and defensive positioning.

3.3 Drone

The surveillance component divides the battlefield into six rectangular scan zones and cycles through them on a fixed route, updating a map of last-known enemy (or friendly) positions. The model assumes a rectangular grid whose width and height are imported from configuration. This is visually represented in Figure 3.1 as the blue (friendly) and red (enemy) rectangles. The grid is partitioned into $n_{cols} \times n_{rows}$ equal rectangles by uniform widths and heights derived from the configured dimensions. The controller precomputes a traversal order over these zones. By default, the friendly controller scans zones [3,4,5,2,1,0] and the enemy controller scans [2,1,0,3,4,5]. This route is hard-coded for six zones and thus implicitly assumes the default 3×2 partition.

Initialization records the controlling “side” and the “target side” to observe, the partition shape, the dwell time per zone, and the per-candidate spotting probability. A simple geometric predicate determines whether a unit lies inside the active zone by comparing its (x,y) position to that zone’s bounds. The spotting routine filters the simulation’s list of units from the target side to those currently inside the active zone and alive. If none qualify, the routine returns immediately. Otherwise, it samples a number of sightings k from a Poisson distribution with mean $\lambda = spot_prob \cdot N$, where N is the number of candidates, truncates k to N , and draws that many distinct units without replacement. This construction increases expected sightings with local density while enforcing that no more units are “spotted” than are present. Each selected unit’s name is recorded with its present coordinates in the drone’s last-known map and transmitted to every friendly unit.

The update loop runs once per simulation tick. It selects which population to observe based on the configured target side, identifies the current zone from the precomputed route, and applies the spotting routine to that zone’s bounds. The controller then increments the dwell counter and upon reaching the configured dwell time, the counter is reset and the route index is advanced cyclically to the next zone. Movement is thus periodic and independent of what is or is not spotted in a given zone. The parameters that have been used can be seen in Table 3.2.

3.4 The HTN Planner & Domains

This section describes the declarative layer of the decision-making system: the HTNs and domain rules that define available missions, their decomposition logic, and doctrinal constraints. Each domain specifies how complex objectives are reduced to primitive

Name	Default	Role
GRID_WIDTH, GRID_HEIGHT	100, 75	Battlefield dimensions used to derive zone widths/heights
n_cols, n_rows	3, 2	Partition shape; total zones $= n_cols \times n_rows$
stay_rounds	10	Ticks to dwell in a zone before advancing the route
spot_prob	0.2	Per-candidate factor setting $\lambda = \text{spot_prob} \times N$ for Poisson sightings
route (friendly)	[3, 4, 5, 2, 1, 0]	Zone visitation order for friendly-side drone
route (enemy)	[2, 1, 0, 3, 4, 5]	Zone visitation order for enemy-side drone
rounds_in_area	0	Dwell counter; resets on zone change
last_known	empty map	Memory: unit name $\rightarrow$ most recent observed (x, y)

TABLE 3.2: Drone parameters

tasks, providing the structural foundation for the runtime controllers discussed later in Section 3.5.

The planner follows a top-down expansion process in which high-level *missions* decompose into ordered rules. Each rule is guarded by a precondition and expands into either primitive actions or further subgoals. Given a task and the current world state, the planner selects the first rule whose precondition is satisfied, expands the associated subtasks, and recurses until only primitive actions remain. Tasks not listed in the domain are treated as primitive and returned as single actions.

A limitation of this deterministic structure is that rule order constrains exploration, meaning that once a guard is satisfied, no alternative branches are evaluated. To provide adaptivity, a Genetic Algorithm (Section 3.7) augments the planner by searching across alternative rule orderings and tactical variations. This hybridization enables the system to explore diverse strategies while remaining attached to doctrinally defined behaviors.

3.4.1 Friendly Domain

The *Friendly Domain* encodes offensive doctrine oriented toward capturing objectives through coordinated staging, flanking, and consolidation. See Figure 3.4 for a representation of the friendly domain. The top-level mission, *Secure Outpost*, expands into a sequence of conditional attack and capture rules. The central precondition for the capture rule is that all known enemies have been defeated; only then can the goal be finalized. If this condition is not met, the planner selects from a hierarchy of offensive operations.

Two primary attack patterns are defined: the *FlankAttack* and the *ConsolidateAttack*. Preconditions determine which branch is selected based on enemy visibility, line-of-sight availability, and relative positioning. When neither attack type is admissible, the plan falls back to a target-engagement rule that prioritizes known enemies by proximity. If no applicable rule can be satisfied, the planner defaults to a *HoldPosition* task, preserving stability until new information becomes available.

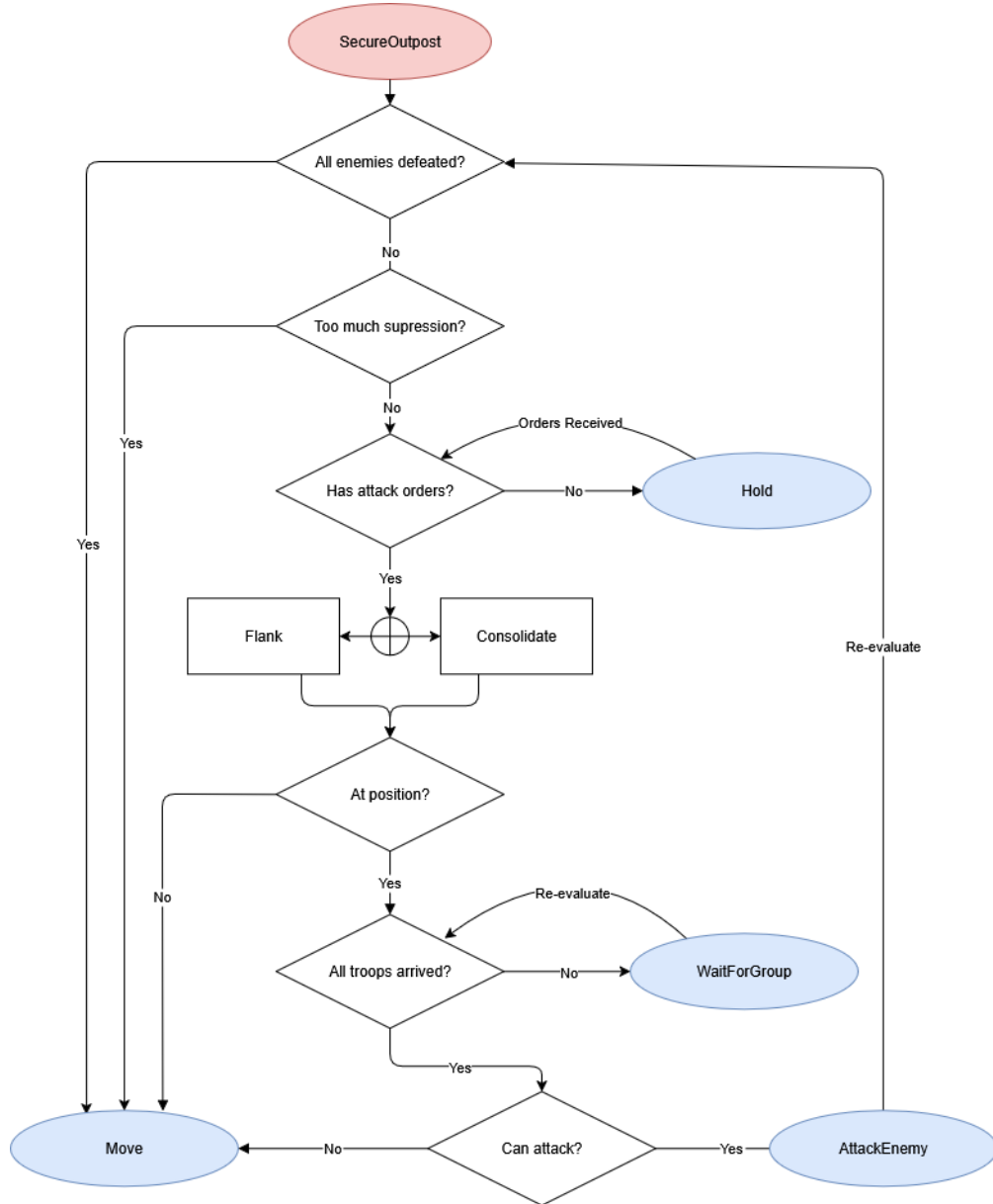


FIGURE 3.4: Flowchart illustrating the rule hierarchy of the Friendly Domain. Diamond-shaped nodes represent preconditions; blue ovals denote primitive tasks.

Multi-enemy phases are generated by a dedicated expander that processes the global attack sequence, represented as `[enemy_id, attack type, assigned friendlies]`. Each phase decomposes into subtasks specifying staging or flanking maneuvers followed by a coordinated assault. Coordination rules ensure that participating agents synchronize

before advancing, while shared variables maintain phase-specific staging coordinates. Within each group, the “front” role is assigned to the unit with the greatest frontal armor value; this role seeds the shared staging point and defines reference geometry for flanking offsets.

Flanking phases decompose into lateral maneuvers relative to the designated front role, while consolidation phases restrict actions to frontal engagement. If flanking geometry cannot be resolved due to spatial constraints, the planner substitutes a frontal assault task. This logic guarantees that the planner always produces a valid tactical configuration even when ideal flanking routes are unavailable.

To preserve perceptual consistency, expansion rules incorporate an observation requirement. Attack phases are deferred until the corresponding enemy has been detected either directly or through sensor memory. This prevents speculative movement toward unconfirmed targets and maintains coherence between world perception and plan expansion.

3.4.2 Enemy Domain

The *Enemy Domain* encodes defensive doctrine based on the *DelayMission* template, which aims to slow the friendly advance so that the step budget expires before the outpost can be captured. The mission decomposes into a sequence of guarded subtasks controlling repositioning, defense, and engagement behaviors.

At the top level, the *DelayMission* checks two global conditions: whether all friendly units have been neutralized and whether the time budget has elapsed. If neither condition is true, the mission continues by selecting from the following rule hierarchy:

- **Retreat rule:** activated when unit health drops below a predefined threshold; expands into repositioning and regrouping subtasks.
- **Defend rule:** selected when the unit is uninjured and not entrenched; decomposes into *PickDefendPosition*, *MoveToPosition*, and *Entrench*.
- **Engage rule:** triggered when line-of-sight to a hostile unit exists; decomposes into *FaceEnemy* and *Attack*.
- **Hold rule:** a fallback action executed when no other preconditions apply.

Figure 3.5 illustrates this hierarchy, showing decision nodes for health, entrenchment, and visibility, and primitive tasks such as moving, facing, attacking, or entrenching. The

structure ensures that enemy forces transition between postures based on explicit doctrinal conditions rather than reactive heuristics, allowing behavior to remain interpretable within the same HTN framework as the friendly side.

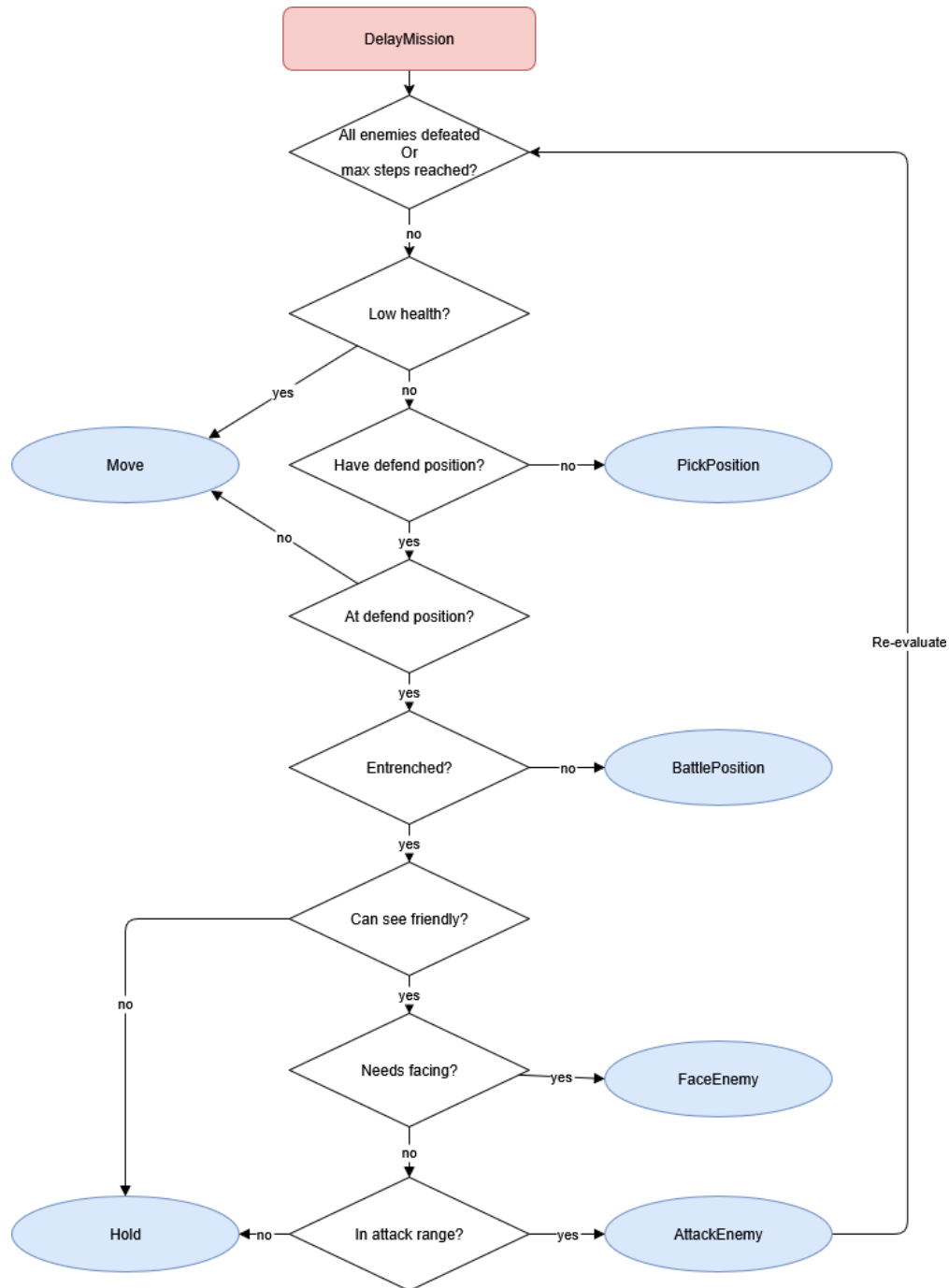


FIGURE 3.5: Flowchart of the DelayMission in the Enemy Domain. Decision nodes denote preconditions; blue ovals denote primitive tasks.



3.5 Friendly and Enemy Units

This section describes how the hierarchical task networks defined in Section 3.4 are executed at runtime by both friendly and enemy controllers. Each unit maintains a queued-task representation of its current plan and updates it every simulation tick in response to evolving battlefield conditions. The active task is evaluated against local preconditions such as visibility, range, suppression, and path availability. When a precondition fails, the controller either skips the step, substitutes an alternative primitive, or triggers a partial replanning cycle. Both sides share identical perception, navigation, and pathfinding systems, ensuring that decisions are made under comparable environmental constraints while differing only in mission objectives and doctrinal parameters.

To improve computational efficiency, identical units were aggregated into homogeneous groups rather than modeled individually. Each group represents a formation of equivalent platforms acting cohesively under a single controller; for instance, twelve tanks are modeled as one *tank group* whose attributes scale with group size. Group-level parameters such as health and combat effectiveness were computed proportionally to the number of entities represented. This abstraction preserves relative combat power while reducing the number of simulated agents, ensuring that coordination and perception routines remain tractable at larger scales.

Unit attributes, including health, firepower, and engagement range, were parameterized using reference ratios derived from the tactical simulation game *Regiments* [32]. While not a primary data source, its internal balance model provides plausible scaling relationships among modern combined-arms units, ensuring consistent relative performance across tanks, infantry, artillery, and anti-tank teams. The full specification of group sizes and parameter values is provided in Appendix A.

3.5.1 Friendly Units

Friendly controllers instantiate the offensive logic defined in the Friendly Domain (Section 3.4) and execute it through a reactive queue of primitive actions. Plans are cached as sequences of tasks, either simple action labels or action–target pairs, and evaluated step by step. At each tick, the controller verifies that the next action remains valid given the current perception and state. If not, it adapts locally without invoking a full replanning cycle. This allows the system to remain responsive to changing contact, terrain, or synchronization constraints.

Tactical Movement and Coordination

Movement encompasses advancing toward the outpost, repositioning, and engaging designated enemies. Outpost coordinates are retrieved from the mission state, while enemy-directed movement first verifies target validity before execution. If the target is visible and within attack range, the queued movement task is rewritten into an attack; otherwise, the controller generates an A* path toward the target using terrain-aware cost penalties for threats and (See Table 3.1). Paths are recomputed every ten ticks to maintain adaptivity. Staging and flanking coordinates are retrieved from the shared cache generated by the phase expander in the Friendly Domain, ensuring consistency between planning and execution layers. If no valid coordinate or path exists, the task is skipped and the controller advances to the next instruction.

Synchronization is achieved through a waiting mechanism. Units pause at staging or flanking positions until all teammates assigned to the same phase have arrived. Only when all synchronization flags are satisfied does the attack phase begin. If required allies are absent or earlier enemies remain unresolved, the unit defaults to a holding state until the condition clears. This design maintains coherent group movement without global locking or excessive replanning overhead.

Reactive Updates and Special Cases

Plans are refreshed when forced by mission-level events, when no valid queue remains, or when immediate threats invalidate the current path. Suppression directly modifies accuracy, and once accuracy falls to zero, the unit temporarily retreats to recover before resuming queued tasks. The first engagement of a plan triggers a broadcast to allied controllers, prompting partial updates to maintain coordination across groups. Specialized rules apply to certain unit types. For instance, artillery can fire indirectly when spotting data is available from drones or other observers.

3.5.2 Enemy Units

Enemy controllers execute the defensive logic of the *DelayMission* defined in the Enemy Domain. Each unit maintains its own queue of primitive actions and evaluates preconditions at every simulation tick. The defensive doctrine prioritizes holding ground, preserving survivability, and delaying the friendly advance within the step budget.

Defensive Positioning and Combat

At initialization, each enemy selects a defensive position derived from mission templates (typically a bridge or predefined defense line) and navigates to it using the same A* pathfinding system as friendly units. Once in position, the unit enters a defensive posture in which armor effectiveness gradually increases while stationary, with additional bonuses when entrenched. During combat, the controller continuously evaluates line-of-sight to visible friendlies, and if a target is detected within weapon range, the unit reorients toward the threat and attacks; if not, it maintains its defensive stance. Health-based rules trigger retreat or reposition actions, directing the unit toward fallback points or the next defensive line. This ensures that positions are not abandoned prematurely while enabling gradual, attrition-based withdrawal.

Reactive Updates and Rotation

The delay mission incorporates rotation rules that reassign defensive responsibilities when predefined thresholds are crossed. When health falls below set ratios, the unit shifts to a secondary defend line or assumes a reserve posture, reinforcing the overall delay effect. Opportunistic reactions, such as immediate facing and firing when a new target appears, override queued holding actions to prevent tactical idleness. Suppression affects firing accuracy and can trigger temporary retreats before re-engagement. Enemy unit types differ only in firing constraints: infantry, tanks, and anti-tank weapons require direct line-of-sight, while artillery can perform indirect fire when spotting data is available. Because perception, navigation, and pathfinding are shared between both sides, any performance asymmetry arises solely from doctrinal parameters rather than environmental advantages.

3.6 The Simulation



The simulator integrates terrain modeling, reconnaissance, unit control, planning, and visualization within a unified tick-driven loop. During initialization, it instantiates friendly and enemy units and links them to a shared world state that includes terrain masks for rivers, forests, and forest edges. Friendly units receive additional state variables such as references to any active enemy, a registry of spotted enemies, and assigned reconnaissance assets. Enemy units are initialized similarly but also precompute a set of westward defensive lines for fallback behavior. Units tasked with a delay mission are given a predefined bridge rotation, health thresholds at 75% and 50% to trigger defensive repositioning, and an initial defend position corresponding to their first assigned

bridge. Both the current and defend positions are set to this location, ensuring that delay units begin the simulation already in a defensive posture.

3.6.1 Behavior Scheduling

Each simulation tick resolves in two phases. First, enemies update: living enemies refresh their plans via the HTN domain, process visible threats, and execute exactly one task from their queue. Defensive phasing logic is applied at this stage. When health thresholds of 75% or 50% are crossed, the first damaged unit triggers a coordinated retreat to the next defensive line, with peers assigned positions relative to that anchor. This ensures that the enemy force falls back cohesively rather than dispersing individually.

After enemy updates, friendlies execute. Each unit first refreshes its perception: line-of-sight checks, cover, and stealth modifiers determine which enemies are visible, and the nearest valid target is cached for quick access. Friendlies then execute one task from their queue, which may involve movement, staging, waiting, or attack depending on current plan state. Synchronization ensures that coordinated phases, such as consolidations or flanking maneuvers, only begin once all assigned units have reached their positions.

3.6.2 Plan Evaluation

Each simulation episode produces a scalar fitness score that quantifies plan effectiveness. The score integrates friendly and enemy health, elapsed steps, and mission success into a single normalized value. Friendly and enemy aggregate health values are divided by their respective maxima to yield dimensionless ratios, and the step count is normalized by the configured maximum. A linear combination rewards friendly survivability, penalizes remaining enemy strength, and applies a time penalty for prolonged engagements. An additional binary term rewards successful capture of the outpost or penalizes failure. The formulation is given in Equation 3.5.

$$\text{Score} = w_f \cdot \frac{H_{\text{fr}}}{H_{\text{fr}}^{\text{max}}} - w_e \cdot \frac{H_{\text{en}}}{H_{\text{en}}^{\text{max}}} - w_t \cdot \frac{t}{t_{\text{max}}} + 10 \cdot \sigma, \quad \sigma \in \{-1, +1\}. \quad (3.5)$$

Here w_f , w_e , and w_t are constant weights applied to friendly health, enemy health, and time respectively. The binary term σ equals +1 if the outpost has been secured and -1 otherwise, ensuring that mission completion dominates the score. This per-episode fitness function serves as the core evaluation used by the genetic optimizer described in Section 3.7. Aggregated performance measures and cross-seed statistics are discussed separately in Section 3.8.2.

3.7 Genetic Optimization

This module implements a population-based optimizer that combines a single-objective genetic algorithm with a multi-objective Non-Dominated Sorting Genetic Algorithm (NSGA-II) variant to search for the optimal force composition and attack sequencing. The design emphasizes correctness (shape and bounds validation), robustness to stochasticity (multi-seed evaluation with variance penalization), and runtime performance (memoization, pre-deduplication and persistent process pools).

3.7.1 The Search Space

The search space is defined by a set of available friendly unit groups and a fixed set of enemy groups. Each friendly entry contains its human-readable name, the class used to instantiate units, a state template, and an integer cost. Four templates are provided in this thesis, namely tanks, infantry, artillery, and anti-tank. The two central data containers are Chromosome and Evaluation. A Chromosome encodes an ordered attack sequence pairing each enemy with an attack type and a selected team of friendly unit names. Evaluation stores the per-chromosome statistics: the mean raw score across seeds, its standard deviation, the adjusted score (mean minus λ times the standard deviation), and the scalarized cost.

3.7.2 GA Environment Initialization

The genetic algorithm begins by constructing the initial simulation environment, which defines the adversarial and friendly forces used for evolutionary evaluation. Enemy units are instantiated in hard-coded formations: infantry, anti-tank, and artillery groups. With each assigned the *DelayMission* objective. Their placement across the map follows predefined spatial distributions to maintain consistent defensive scenarios across trials.

Friendly forces are generated dynamically from configurable templates that specify allowable unit counts for each type. For every template (infantry, tank, artillery, or anti-tank), a random number of units within the permitted range is instantiated and bound to the simulation domain. Each chromosome encodes two main components: a friendly force configuration and an ordered attack plan. The plan is represented as a sequence of engagements, where every enemy target is paired with a selected subset of friendly units and an attack mode, either *FlankAttack* or *ConsolidateAttack*.

Initial populations are formed through randomized sampling. Enemy orders are shuffled, attack types are drawn at random, and friendly teams are selected using a bias that

slightly favors doctrinally appropriate matchups, where anti-tank and tank units are more likely to be paired with armored targets, while infantry and artillery tend to be assigned against soft positions. This introduces a weak heuristic that improves early search efficiency without constraining exploration.

Before evaluation, each chromosome is checked for validity. Counts exceeding template bounds are clamped, missing assignments are repaired, and duplicates are removed through canonical hashing of configuration and team composition. The resulting population provides a diverse and doctrinally coherent starting point for evolutionary optimization under the shared simulation environment.

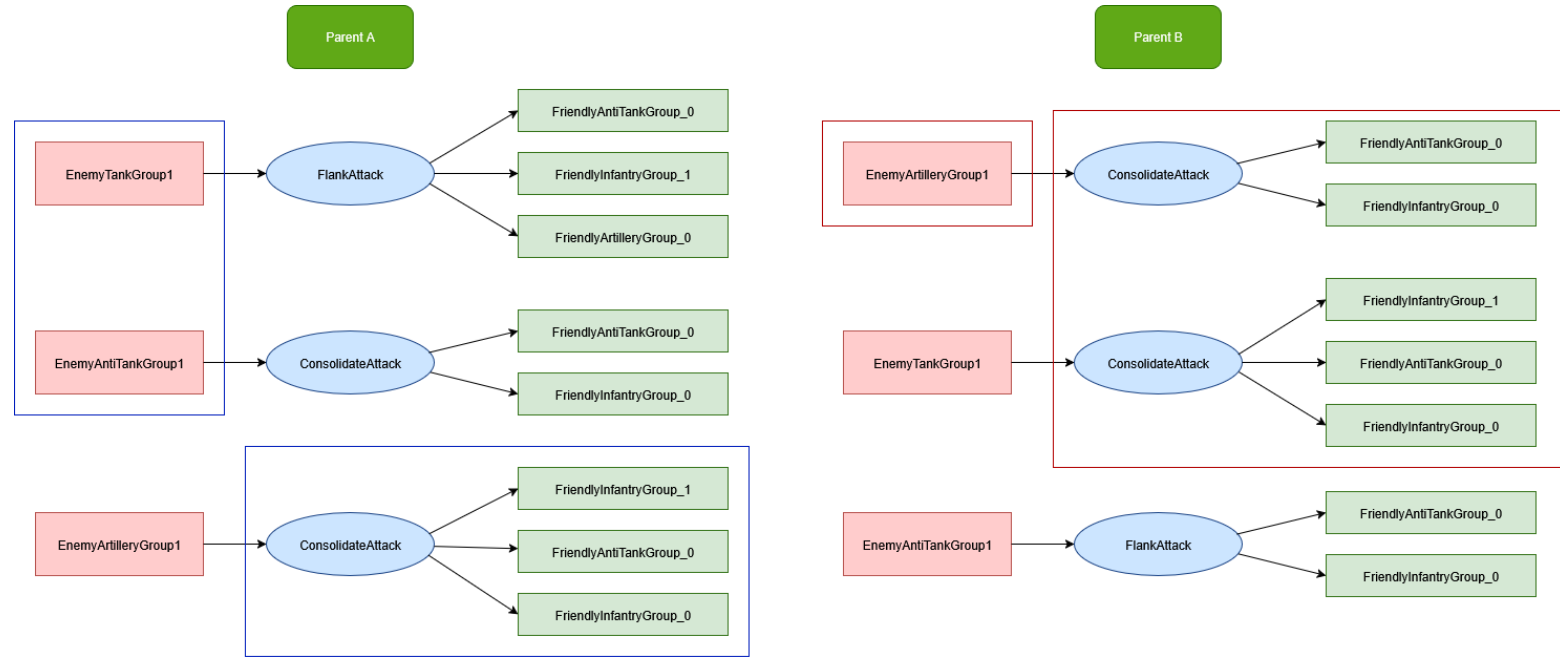
3.7.3 Evaluation

A single run of the GA constructs the friendly and enemy units and creates a Simulation instance, optionally injecting a seed into any seedable attributes present. The chromosome’s attack sequence is then assigned to the simulation and executed up to a maximum number of steps, returning a scalar “score”. Multi-seed evaluation repeats this process for a fixed number of seeds with a deterministic stride ($seed_base + 9973 \cdot j$) to reduce accidental correlations. From these runs the evaluator computes the mean and population standard deviation of the raw score, and then the robust adjusted score $mean - \lambda \cdot std$, where λ is a user-set robustness parameter (see Section 3.6.2). In parallel, the evaluator computes a linear cost as the dot-product between per-template counts and template costs. In single-objective mode, fitness is the adjusted score penalized by cost with a user-tunable weight; in NSGA-II, adjusted score (maximize) and cost (minimize) form the two objectives.

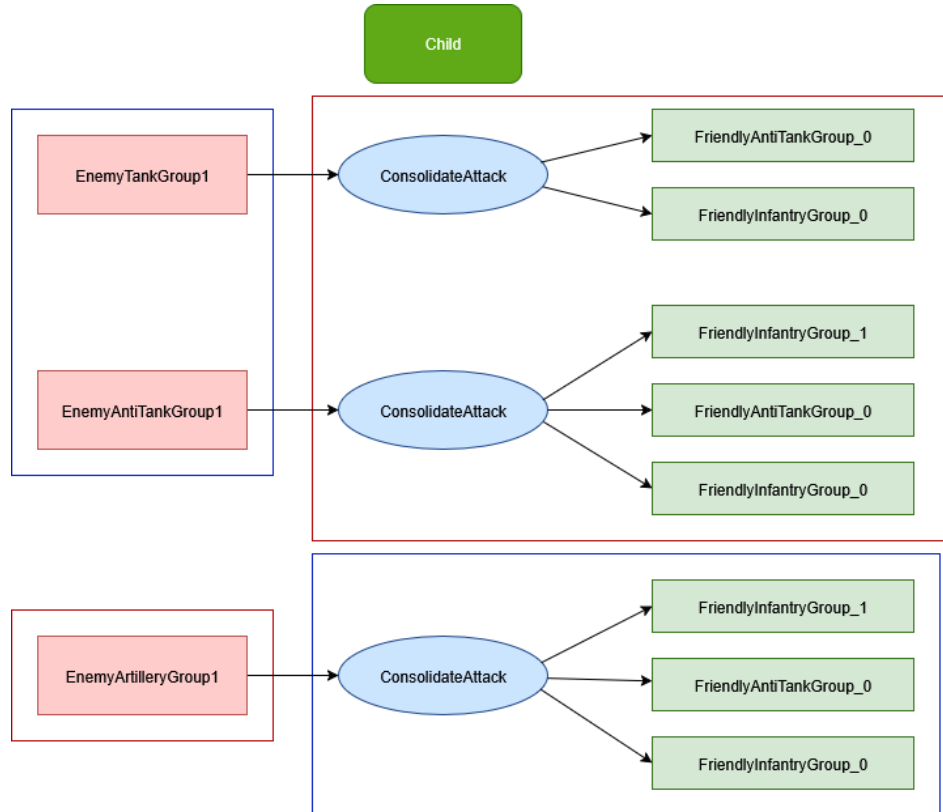
3.7.4 Crossover and Mutation

Variation operators modify both composition and sequence while preserving validity. As can be seen in Figure 3.6, the crossover operator performs an order-preserving recombination of the engagement sequence by cutting one parent’s order and filling the remainder from the other. Then, for each engagement, choosing whether to inherit the attack type and team from the parent A or parent B.

After crossover, mutation introduces additional variation into the child chromosome. A mutation is applied whenever a pseudo-randomly generated number falls below a pre-defined probability p_{mut} . This probability is scheduled to decrease over generations, beginning at a relatively high value and annealing to a lower value, with the exact parameters determined by hyperparameter optimization (see Section 3.8). Three mutation



(A) Parent sequences used for crossover.



(B) Child sequence generated by crossover.

FIGURE 3.6: Illustration of crossover: (a) parent sequences are partially inherited and (b) recombined into a new child plan.



types are used: (i) a block-shift move, which selects a contiguous segment and reinserts it at a random new position in the sequence (Figure B.1); (ii) flipping the attack type of an engagement (Figure B.2); and (iii) resampling the friendly team assigned to an engagement, either according to configured biases (e.g., anti-tank units against tanks) or uniformly at random (Figure B.3).

3.7.5 Selection

Two selection regimes are supported. For single-objective optimization, the algorithm uses batched tournament selection over scalar fitness to choose next generation. For the multi-objective case, the implementation follows the NSGA-II pipeline: it defines Pareto dominance with respect to adjusted score (to maximize), cost (to minimize), applies the fast non-dominated sort to obtain fronts, computes crowding distance separately on the two objectives (with $\pm\infty$ for boundary points in each objective), and performs environmental selection that fills the next generation front by front, tie-breaking with crowding distance in descending order. The procedure returns the truncated population.

3.7.6 Generational Optimization Loop

The optimization loop orchestrates the entire evolutionary process, combining population management, parallelized evaluation, and adaptive variation. Given enemy compositions, population size, generation count, and per-template count bounds, the procedure initializes an ordered-dictionary-backed least-recently-used (LRU) cache to store evaluation results, keyed by canonical chromosome representations. Cache hits fully bypass recomputation, significantly reducing runtime when structurally similar individuals recur across generations.

To exploit multi-core systems, evaluation is distributed through a pickle-safe top-level worker function. The evaluator canonicalizes all chromosomes, deduplicates identical entries within the batch, and dispatches only the unique remainder to a persistent process pool. Results are then written back to the cache and re-broadcast to their original indices, ensuring alignment between evaluation outputs and the current population.

If a checkpoint path is provided, optimization resumes from that state; otherwise a new population is initialized and evaluated. Across generations, the algorithm applies linearly annealed mutation probabilities, gradually shifting from exploration to exploitation. In NSGA-II mode, each generation forms elites from the previous Pareto front, generates offspring through crossover, mutation, and repair, and filters duplicates by

canonical key. Any shortfall in population size is filled with random but valid individuals. Environmental selection then determines the next generation, while progress logs record front size and score ranges within the Pareto set.

In single-objective mode, the loop performs tournament selection to build a mating pool, applies variation operators as above, and evaluates the resulting offspring. After each generation, checkpoints may be saved for recovery or long-term experiments. Upon completion, the optimizer optionally re-evaluates the top individuals under multiple seeds to obtain cleaner performance estimates. The final output includes the best individuals overall and per generation, the Pareto front (if applicable), per-generation summary statistics, and the fully evaluated final population.

3.8 Hyperparameter Optimization

This module performs a two-stage hyperparameter search around a GA. The approach uses fast iteration with automatic checkpointing and a confirmatory rerun at higher fidelity. It samples a small set of GA knobs from bounded ranges, evaluates each configuration by running the optimizer with a fixed scenario, and ranks results using either a scalar fitness (single-objective mode) or a hypervolume indicator over score–cost tuples (NSGA-II mode).

3.8.1 Parameter Sampling

Parameter sampling defines the configuration space explored during hyperparameter optimization. A local pseudo-random generator ensures reproducibility while allowing each trial to draw a distinct combination of search parameters. For every configuration, the sampler selects values for the initial and final mutation probabilities, the chance of applying a block-shift mutation, the number of elites retained each generation, the fraction of random individuals introduced for diversity, and the robustness and cost-scalarization weights. Mutation probabilities are constrained such that the ending value never exceeds the starting value, producing a natural annealing effect that gradually shifts the balance from exploration toward exploitation across generations.

3.8.2 Evaluation

Experimental evaluation aggregates the per-simulation fitness scores defined in Section 3.6.2 into higher-level performance metrics. Each optimization trial produces multiple stochastic outcomes, which are averaged and shaped to reflect both quality and

robustness. Two complementary criteria are used: a scalar adjusted score and a multi-objective hypervolume measure.

For a single simulation, the raw scalar score s is computed as in Equation 3.5. To improve robustness, this value is averaged across multiple random seeds and adjusted with temporal and variance penalties:

$$s_{\text{adj}} = \bar{s} - w_t \left(\frac{t}{t_{\text{max}}} \right) - \lambda \sigma_s, \quad (3.6)$$

where $\bar{s}$ is the mean score, σ_s its standard deviation, w_t the time penalty, and λ a robustness coefficient.

Each configuration incurs a cost based on the composition of friendly forces. Table 3.3 lists the relative weights assigned to each unit type, which serve as cost coefficients in Equation 3.7.

$$c = \frac{1}{C_{\text{max}}} \sum_i n_i \text{cost}_i. \quad (3.7)$$

In NSGA-II mode, (s_{adj}, c) define a two-dimensional Pareto objective. The hypervolume indicator then measures the space dominated by the resulting Pareto front relative to a reference point beyond the worst observed values, capturing both solution quality and diversity. When a Pareto front is absent (e.g., in single-objective mode), the best scalar fitness from the final generation is used as a fallback metric.

Trials were executed through a unified runner that handled checkpointing, seed initialization, and fidelity control. Low-fidelity runs used smaller populations and generations, while high-fidelity runs doubled both and averaged results across seeds. Each trial returned the final metric, runtime, and configuration data for reproducibility.

Parameter	Value	Role
w_t (time penalty)	proportional to step fraction	Reduces adjusted score for longer simulations
λ (robustness penalty)	1.0	Penalizes variance across seeds
Tank cost	5	Relative cost per tank unit
Artillery cost	4	Relative cost per artillery unit
Anti-tank team cost	3	Relative cost per anti-tank team
Infantry cost	2	Relative cost per infantry unit
Reference point	slightly beyond worst values	Defines hypervolume integration limit

TABLE 3.3: Parameters used for adjusted score and cost-based evaluation.

3.8.3 Main Routine

The main routine defines a two-stage optimization schedule designed to balance exploration and computational cost. In Stage A, a limited number of low-fidelity trials are conducted. For each trial, the procedure samples a new set of hyperparameters, assigns a distinct random seed from an additive ladder, executes the optimization, and records both the achieved metric and runtime. The top- K configurations from Stage A, ranked by performance, advance to Stage B. These candidates are re-evaluated under higher-fidelity settings using independent seed ladders to minimize correlation. The configuration yielding the highest confirmed metric is selected as the final result. If Stage B is skipped or yields no valid confirmations, the best-performing low-fidelity configuration is retained instead.

Chapter 4

Experiments and results

4.1 Effect of HPO on Performance

This experiment examines how HPO influences the performance and robustness of the genetic algorithm across varying fidelity levels. The methodology for scoring and evaluation follows the definitions in Section 3.8.2. Here, the emphasis is on comparative outcomes rather than implementation details.

The search was executed in two stages. An initial low-fidelity sweep identified promising hyperparameter regions, and a subsequent high-fidelity confirmation stage re-evaluated the top-ranked configurations under extended budgets. Each configuration was assessed using the NSGA-II objective pair—adjusted score (to be maximized) and cost (to be minimized)—with overall performance summarized by the hypervolume indicator. When a Pareto front was absent, the best final-generation scalar fitness served as a fallback metric.

Low-fidelity runs employed smaller populations and fewer generations, while high-fidelity confirmations doubled both parameters. A consistent seed ladder ensured reproducibility across stochastic elements. The environment retained a fixed structural layout (river and bridge topology, enemy roster) while forest distribution and local engagements remained stochastic, introducing tactical variability between seeds. This combination preserved experimental comparability while retaining realism in combat dynamics.

Table 4.1 summarizes the key HPO configurations used for these experiments. The results reported in the following sections analyze how parameter tuning affected convergence speed, final fitness distribution, and Pareto-front quality across optimization modes.

Argument	Default	Role
mode	nsga2	GA style: multi-objective NSGA-II or single-objective
low_trials	4	Number of low-fidelity trials (Stage A)
confirm	2	Number of top configs to confirm at high fidelity (Stage B)
low_generations, low_pop	5, 10	Low-fidelity GA budget (gens, pop)
high_generations, high_pop, high_seeds	10, 20, 3	High-fidelity GA budget and eval seeds
workers	8	Parallel evaluation workers
max_steps	500	Per-run simulation step budget
min_team_size	2	Lower bound on total friendly team size
max_count	2	Per-template upper bound for friendlies
worker_log	eval_runs.log	Worker print sink
mutate_p_start_min, _max	0.3, 0.7	Range for initial mutation probability
mutate_p_end_min, _max	0.02, 0.25	Range for final mutation probability (capped $\leq$ start)
block_shift_min, _max	0.2, 0.8	Range for block-shift mutation probability
elites_min, _max	2, 2	Range for elites per generation (fixed at 2 by default)
random_frac_min, _max	0.0, 0.3	Range for random individuals per generation
robust_lambda_min, _max	0.0, 2.0	Range for robustness weight across seeds
cost_weight_min, _max	0.0, 3.0	Range for cost weight (unused in NSGA-II mode)
seed	123	Base RNG seed for parameter sampling and trial ladders
ckpt_every	0	Checkpoint cadence in generations (0 = off)
ckpt_dir	hpo_ckpts	Directory for per-trial checkpoints
save_best	False	Save per-trial best_*.pkl artifacts
resume_trials	False	Auto-resume trials if a matching checkpoint exists

TABLE 4.1: Arguments passed to the HPO driver and their roles.

4.1.1 Fitness Scaling and Normalization

Scalar fitness values reported by the GA are derived from the adjusted score defined in Section 3.6.2. This value incorporates penalties for runtime and unit cost, and it serves as the single optimization target during training. The adjusted score is already

normalized within $[0, 1]$, but further scaling is applied to express results on a $[0, 100]$ scale for readability.

Under ideal conditions (complete enemy elimination and full friendly survival) the two evaluation regimes yield different theoretical maxima. Without a time penalty, the ceiling score is

$$100 \times 1.0 \times \left(1 - \frac{2}{42}\right) \approx 95.2, \quad (4.1)$$

whereas applying a time penalty of 0.5 halves this value to

$$100 \times 0.5 \times \left(1 - \frac{2}{42}\right) \approx 47.6. \quad (4.2)$$

Consequently, raw fitness values from the two regimes are not directly comparable. The results reported in subsequent sections therefore use normalized scores to allow consistent interpretation across experimental conditions.

4.1.2 Low-Fidelity Trials

No Time Penalty

Table 4.2 summarizes the outcomes of the four exploratory runs without any time penalty. Each configuration used 10 individuals for 5 generations. Hypervolume values ranged from 2.0 to 9.0, showing a clear separation between lower- and higher-performing configurations. Trial 3 achieved the best low-fidelity result, nearly doubling the hypervolume of the second-best trial. This configuration combined a relatively high initial mutation rate with a strong annealing schedule, while also maintaining a balanced block-shift probability and a small random-injection fraction.

Trial	HV metric	p_{start}	p_{end}	Block-shift	Random frac
0	2.0	0.321	0.040	0.444	0.252
1	4.0	0.525	0.098	0.231	0.101
2	5.0	0.301	0.120	0.253	0.021
3	9.0	0.662	0.041	0.285	0.088

TABLE 4.2: Summary of low-fidelity trials without time penalty.

With Time Penalty

When a time penalty (0.5) was introduced, the relative ordering of trials changed (Table 4.3). Hypervolume values now ranged between 11.0 and 24.0. Trial 0 contained the

best configuration, achieving the maximum hypervolume of 24.0, while trials 2 and 3 filled the middle range at 12.0 and 11.0 respectively. Compared to the no-penalty results, the rankings shifted substantially: trial 0, which previously performed only moderately, dominated once runtime was incorporated into the objective. This indicates that the inclusion of time cost reweighted the search toward shorter and more efficient engagement sequences, while some parameterizations that performed better without time costs (e.g., trial 3) were penalized.

Trial	HV metric	p_{start}	p_{end}	Block-shift	Random frac
0	24.0	0.687	0.121	0.204	0.201
1	14.0	0.395	0.027	0.673	0.194
2	12.0	0.337	0.184	0.707	0.264
3	11.0	0.665	0.036	0.605	0.081

TABLE 4.3: Summary of low-fidelity trials with time penalty (0.5).

4.1.3 High-Fidelity Trials

No Time Penalty

The top two candidates from the no-penalty sweep were confirmed with doubled budgets and replicated evaluation (Table 4.4). Both converged to identical hypervolume values of 8.0 and a normalized fitness of 87.5 despite differences in mutation schedule. Trial 3 retained its advantage in selection order due to deterministic tie-breaking.

From trial	HV metric	Pop	Gens	Seeds	$p_{\text{start}} \rightarrow p_{\text{end}}$
3	8.0	20	10	3	0.662 $\rightarrow$ 0.041
2	8.0	20	10	3	0.301 $\rightarrow$ 0.120

TABLE 4.4: High-fidelity confirmation trials without time penalty.

With Time Penalty

Under the time-penalized regime, only trial 0 was promoted and confirmed (Table 4.5). It reproduced its low-fidelity advantage with a hypervolume of 24.0 and final-generation normalized fitness of 86.7.

From trial	HV metric	Pop	Gens	Seeds	$p_{\text{start}} \rightarrow p_{\text{end}}$
0	24.0	20	10	3	0.687 $\rightarrow$ 0.121

TABLE 4.5: High-fidelity confirmation with time penalty.

4.1.4 Final Picks and Comparison

The final configurations from both regimes are summarized in Table 4.6. Without a time penalty, the optimizer selected a strongly annealed schedule with modest block shifts, a low random-injection rate, and a nonzero robustness weight. With time penalty, the optimizer shifted to a higher end-of-run mutation rate, a larger random-injection fraction, and a very weak robustness weight, reflecting a preference for efficiency and cost minimization. The normalized fitness remained comparable, but hypervolume increased due to the time component in the objective¹.

Metric	No time penalty	With time penalty
Final-gen norm fitness	87.5	86.7
Hypervolume (confirm)	8.0	24.0
Mutation schedule	0.662 $\rightarrow$ 0.041	0.687 $\rightarrow$ 0.121
Block-shift prob.	0.285	0.204
Random frac.	0.088	0.201
Robustness weight λ	0.861	0.168
Time weight	0.0	0.5

TABLE 4.6: Comparison of final configurations with and without time penalty.

The two-stage hyperparameter search demonstrated that objective formulation critically shapes outcomes. Without time costs, the optimizer emphasized robustness. With time penalty, it favored faster, lower-cost engagements, trading away robustness and maximum score. This shows the importance of aligning optimization objectives with operational priorities when configuring evolutionary search.



4.2 Verification of Selected Plan

The hyperparameter optimization stage yielded fixed candidate plans that were then re-evaluated to assess robustness under stochastic variation. Each plan was run thirty times under distinct random seeds, without further genetic search or modification of the sequence. Results are reported separately for the no-penalty and time-penalty regimes, since their scalar objectives differ.

¹Hypervolume values quantify the dominated area of the Pareto set over adjusted score (to be maximized) and cost (to be minimized). Higher values indicate better coverage within a given objective formulation. However, values are not directly comparable across regimes (e.g., no penalty vs. time penalty), since the underlying score scaling differs.

4.2.1 No Time Penalty

The plan selected without time penalty consisted of one artillery and one anti-tank group, structured as a four-phase attack sequence. Both units acted jointly throughout: first staging and consolidating against the enemy anti-tank, then flanking the enemy tank and infantry in sequence, and finally eliminating the enemy artillery before securing the outpost. The timing and coordination of this sequence can be seen in the GANTT chart (Figure 4.1). Task colors indicate coordinated phases of staging, movement, attack, and holding, illustrating the doctrinal clarity and reproducibility of the plan across simulation steps.

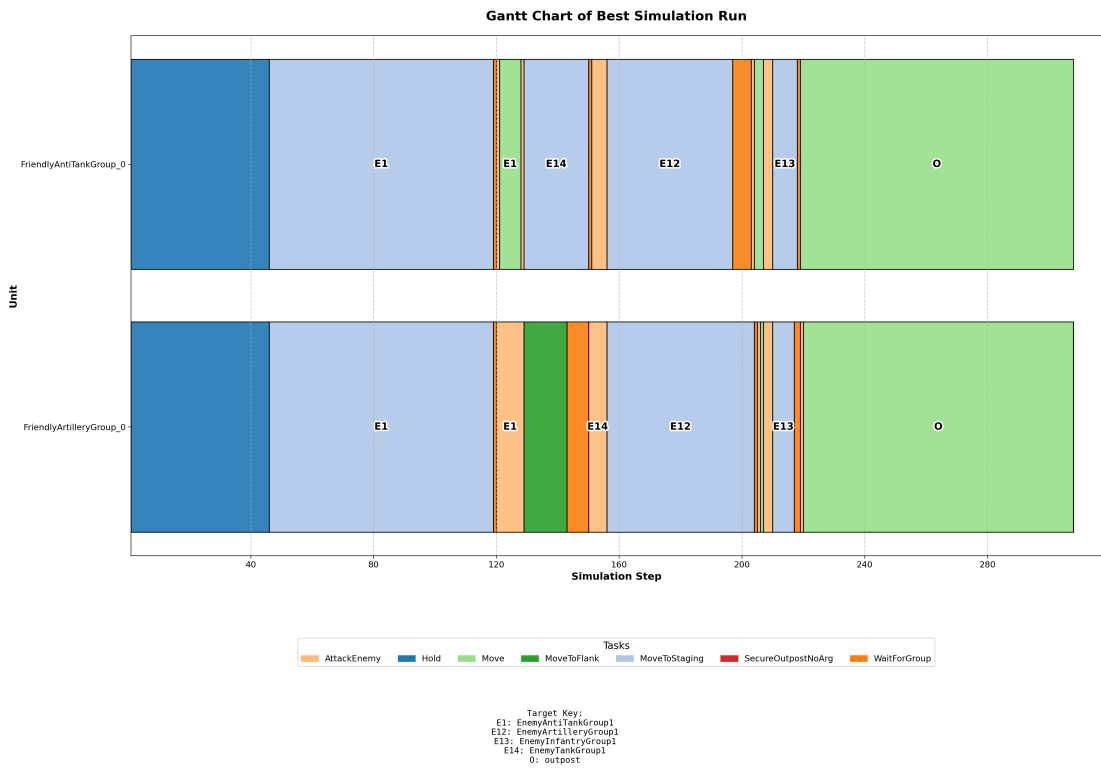


FIGURE 4.1: Gantt chart of the best plan from the verification runs without time penalty.

Across thirty replications the mean score was 20.8 with a standard deviation of 10.7. Success, defined as securing the outpost before termination, was achieved in 29 of 30 runs (96.7%). On average the engagements lasted 326 steps out of the 500-step budget. Normalized against the theoretical maximum of 95.2, the plan's final scalar fitness corresponds to 0.88. These statistics confirm that the plan was effective and resilient to stochastic combat and detection noise.

Seeds evaluated	30
Mean raw score	20.8
Std. deviation (score)	10.7
Success rate	0.967
Mean steps	326
Final scalar fitness (fraction of max)	0.88

TABLE 4.7: Seed dispersion results for the no-penalty plan.

4.2.2 With Time Penalty

The plan selected under a time penalty of 0.5 employed one artillery group and two anti-tank groups, omitting tanks and infantry entirely. Its sequence consisted of four consecutive consolidations: first against the enemy tank with both anti-tank units, then against the enemy anti-tank with both anti-tanks plus the artillery, followed by a consolidation on the enemy infantry with the same combined force, and finally a joint consolidation against the enemy artillery. This produced a compact, force-concentrated strategy emphasizing rapid elimination of each target in turn, consistent with the time-sensitive objective. Despite its efficiency, the plan incurred losses: the friendly *AntiTankGroup_0* was destroyed by enemy artillery fire during the final engagement, as shown in the GANTT chart (Figure 4.2).

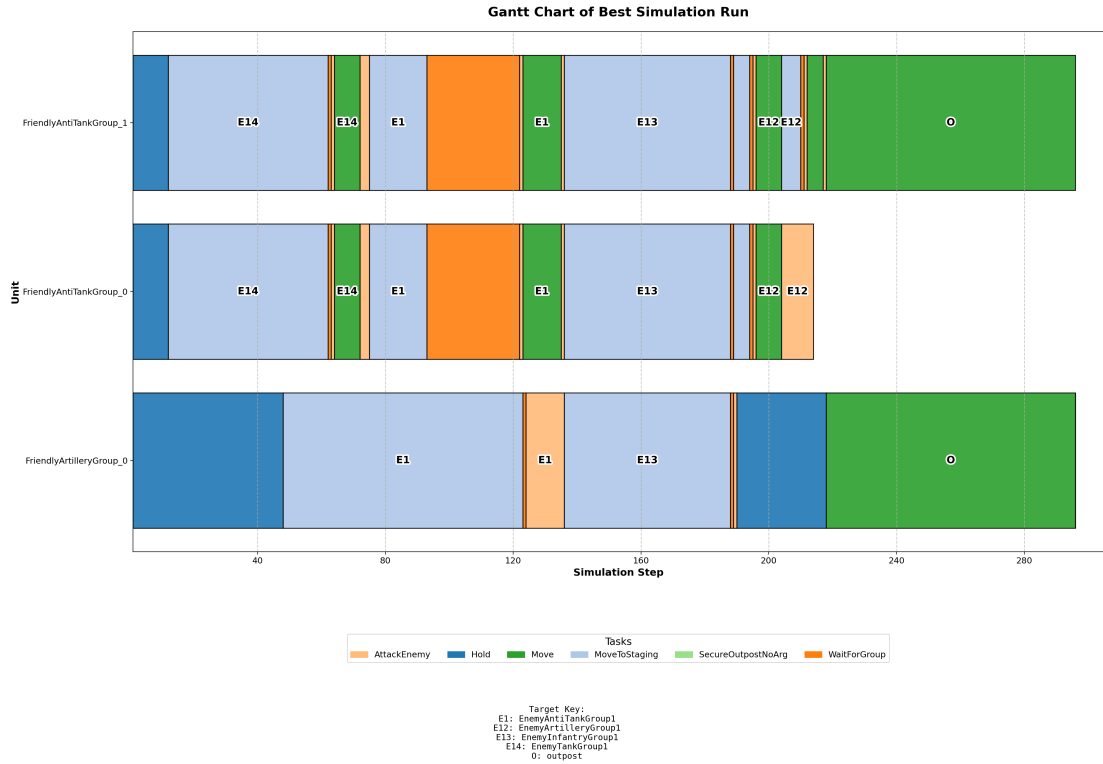


FIGURE 4.2: Gantt chart of the best plan from the verification runs with time penalty.

Across thirty replications the mean score was 1.36 with a standard deviation of 26.0, reflecting a strongly bimodal distribution of outcomes. 60% of runs succeeded with short engagements (≈ 280 – 320 steps), while 40% failed, often running to the 500-step cap. On average, friendly forces retained 60.2 health while enemies were left with 12.1 health. Normalized against the theoretical maximum of 47.6, the plan’s final scalar fitness corresponds to 87.5.

Seeds evaluated	30
Mean raw score	1.36
Std. deviation (score)	26.0
Success rate	0.60
Mean steps	323
Final scalar fitness (fraction of max)	0.87

TABLE 4.8: Seed dispersion results for the time-penalized plan.

4.2.3 Comparison

Both regimes produced compact artillery–anti-tank plans that generalized well beyond their discovery seeds but optimized for different priorities. Without a time penalty, the objective emphasized survival and damage, leading the GA to favor conservative plans that consistently dismantled the enemy roster. These runs often extended close to the 500-step budget yet still scored highly, achieving a 97% success rate since duration carried no cost.

Introducing a time penalty of 0.5 reweighted the objective toward efficiency, prompting the optimizer to discover faster, force-concentrated strategies. The selected plan relied on consecutive consolidations to eliminate enemies rapidly, completing in roughly 280–320 steps. However, this aggressiveness came at the expense of stability: early spotting failures or missed opening shots often led to collapse, lowering overall success to about 60%.

Although both regimes yielded similar mean step counts (326 vs. 323), their outcome distributions reveal a clear robustness–efficiency trade-off. The no-penalty plan achieved slower but dependable victories, while the time-penalized plan produced quicker successes when conditions aligned but failed more often under adverse randomness. Normalized by each regime’s theoretical maximum, both plans performed comparably (≈ 0.88 vs. ≈ 0.87), showing that the apparent score drop in the time-penalized case reflects the stricter objective rather than poorer optimization.

4.3 Transfer and Robustness Analysis

To assess how well the optimized parameters generalize beyond the original training conditions, two alternative enemy rosters were introduced. These variants tested the framework’s robustness under different operational pressures. One emphasizing armored offense and the other relying on indirect fire and infantry mass. Each roster was evaluated under both objective formulations—without and with a time penalty of 0.5—using the same hyperparameters obtained during baseline optimization. This setup isolates the effect of enemy composition on plan quality, revealing whether the tuned GA and HTN integration can adapt to substantially different tactical contexts.



4.3.1 Roster A: Armor Push with AT Overwatch

This enemy roster emphasized sustained armored pressure, with two tanks supported by an anti-tank unit and infantry. It forced friendly anti-tank groups to sequence perfectly, while infantry spotting limited artillery effectiveness.

Without a time penalty, the GA converged on the same compact artillery–anti-tank duo that dominated the baseline scenario. This pair consolidated against one tank, flanked the second, and then eliminated the infantry and anti-tank in sequence. The solution reliably achieved 0.88 of the theoretical maximum in its regime, confirming that the tuned hyperparameters transferred effectively even under increased armored pressure.

With a time penalty, the GA shifted toward broader force compositions. The best plan employed two infantry groups, one artillery, and one anti-tank unit. It flanked the second tank first, consolidated against infantry, and then neutralized the anti-tank and remaining tank. This composition emphasized redundancy and efficiency, reducing reliance on a minimal duo and instead spreading capabilities across multiple groups. Although raw scores were lower under the penalized objective, when scaled by the reduced theoretical maximum the relative quality was again 0.88. The difference lay in that without penalty, robustness favored a minimal but reliable duo, while with time penalty, efficiency favored larger rosters and early flanking.

4.3.2 Roster B: Indirect Fire and Infantry Pressure

The second roster removed heavy armor, instead consisting of two infantry, one artillery, and one anti-tank unit. This composition applied pressure through indirect fire and massed infantry, while the anti-tank remained a high-value target. The challenge was

thus to adapt strategies optimized for defeating tanks to scenarios where armor was absent.

Without a time penalty, the GA once again converged on artillery paired with an anti-tank group, neutralizing the enemy anti-tank first, followed by artillery and then infantry. This plan reached only 0.40 of the theoretical maximum in its regime, indicating that optimization was less effective than in the baseline or Roster A. Failures often arose when spotting or suppression delayed anti-tank elimination, allowing artillery and infantry attrition to accumulate. The optimizer thus demonstrated partial but not complete transfer to infantry- and artillery-heavy contexts.

With a time penalty, the GA favored a much leaner composition. The strongest plan consisted of a single tank and a single infantry group. Together they consolidated against one infantry unit, then executed successive flanking maneuvers against the enemy anti-tank, the second infantry, and finally the artillery. This compact roster achieved 0.50 of the penalized maximum, outperforming the no-penalty variant in relative terms despite its simplicity. Here the optimizer clearly shifted toward efficiency and simplicity under the penalized objective, yielding relatively better normalized outcomes than in the no-penalty regime.

4.3.3 Comparison Across Rosters

Across all three test rosters—the original mixed composition, the armored push of Roster A, and the infantry–artillery pressure of Roster B—the hyperparameters selected during hyperparameter optimization (HPO) maintained effective performance but exhibited distinct emphases depending on the objective formulation.

Without a time penalty, both the baseline roster and Roster A achieved 0.88 of their respective theoretical maxima, indicating that the optimized settings generalized well to scenarios dominated by armored units. In contrast, Roster B reached only 0.40 of its theoretical maximum, showing that performance transfer weakened when the opposing force lacked armor and relied primarily on artillery and infantry.

When a time penalty of 0.5 was introduced, the baseline plan achieved 0.87 of the penalized maximum, while Roster A again reached 0.88, demonstrating that both scenarios preserved their relative optimization quality under stricter efficiency constraints. Roster B improved slightly to 0.50, as the optimizer adapted toward more compact, efficient plans—particularly tank–infantry combinations—that better suited the penalized objective.

Overall, these results (summarized in Table 4.9) show that the HPO-derived parameters consistently supported high-quality optimization in armor-heavy environments, while generalization was less complete in infantry- and artillery-dominated scenarios. The inclusion of a time penalty partially mitigated this effect by promoting leaner and faster plans that achieved relatively better normalized outcomes under their respective objective.

Roster	No time penalty fit.	With time penalty fit.
Baseline (1 of each)	0.88	0.87
Roster A (2 tanks + AT + inf.)	0.88	0.88
Roster B (2 inf. + AT + artillery)	0.40	0.50

TABLE 4.9: Normalized final scalar fitness across rosters and regimes.



Chapter 5

Discussion

The experiments presented in this thesis demonstrate that hierarchical task networks combined with evolutionary optimization can produce doctrinally plausible and interpretable plans under stochastic battlefield conditions. At the same time, it is important to emphasize the assumptions and simplifications underlying the framework, as well as the scope and limitations of the hyperparameter optimization conducted. The approach followed a philosophy of “make it work, make it right, make it fast.” The priority was first to establish feasibility, then to ensure methodological correctness, with performance optimization deferred to future work.

5.1 Assumptions and Data Sources

A large number of assumptions were made in defining the environment, the unit behaviors, and the evaluation metrics. State template values were derived from the commercial wargame *Regiments 3*, rather than from empirical or doctrinal military data. In principle, real-world data exists, but access is constrained by security clearance, making it inaccessible for this research. *Regiments 3* was selected because its developers publicly released their unit parameters, and while the numbers themselves are fictional, they are internally balanced to provide fair interplay between units. This provided a usable starting point, as designing balanced interactions entirely from scratch would have been difficult. The stochastic processes implemented in the simulation (forest generation, spotting events, suppression effects, and variable damage) create meaningful variability across runs, but the behaviors themselves remain stylized abstractions. Results should therefore be interpreted as demonstrations of algorithmic capability rather than as predictions of real-world tactical effectiveness.

5.2 Model Scope and Abstractions

In the current model infantry cannot defeat tanks, even at close range, whereas in real-world combat well-positioned infantry equipped with anti-armor weapons may achieve such outcomes, and it may even be the preferred tactic in urban or forested areas. Addressing this limitation would require enriching the representation of units with additional attributes, expanding the doctrinal templates, and incorporating a broader set of predefined tactics that the optimizer could operate with. Such extensions would make the environment more realistic and would allow evolutionary search to uncover strategies that reflect the full complexity of combined-arms combat.

It should also be noted that the current framework has primarily modeled the *combat* function, focusing on maneuvering and direct engagements. In practice, operational plans also include extensive provisions for *combat support* (e.g., intelligence, reconnaissance, electronic warfare, engineering) and *combat service support* (e.g., logistics, medical evacuation, supply). Incorporating these dimensions would significantly expand the search space, as optimization would need to account not only for tactical sequencing but also for sustainment, reinforcement, and enabling functions. While their omission narrows the scope of the present study, it also keeps the problem computationally tractable. Future work could progressively introduce such elements, though doing so will require more advanced search strategies and careful abstraction to balance fidelity with solvability.



5.3 Integration of HTN and GA

A central strength of the framework lies in the integration of hierarchical task networks with genetic algorithms. HTNs constrained the search to doctrinally valid plan structures, while the GA explored and optimized sequencing and assignments within those structures. This integration proved highly effective: it both preserved doctrinal plausibility and enabled efficient search, avoiding wasted effort on unrealistic plans. The generated plans were compact, interpretable, and often mirrored tactical reasoning that a human commander might employ, such as prioritizing anti-tank elimination before committing to maneuver.



5.4 Alternative Optimization Methods

While Genetic Algorithms were selected as the primary optimization method in this thesis, several other families of metaheuristics could also have been considered. These alternatives differ in their suitability depending on whether the search space is continuous or discrete, deterministic or stochastic, and whether multiple objectives must be balanced.

Close relatives of GAs include Evolution Strategies (ES), Differential Evolution (DE), Memetic Algorithms, and Genetic Programming (GP). ES and DE are most effective in continuous domains, while Memetic Algorithms combine evolutionary search with local refinement to accelerate convergence. GP evolves tree-like procedural structures that could directly encode plans. These methods remain viable, though GAs provided the most straightforward integration with hierarchical task networks [33–35].

Nature-inspired approaches such as Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO), Bee Colony Optimization, Firefly, and Bat Algorithms offer alternative exploration strategies. PSO is well suited for continuous optimization, whereas ACO excels in routing and sequencing problems. However, most swarm and colony-based methods assume homogeneous agents with identical behaviors, making them less compatible with doctrinally constrained and heterogeneous force models [36].

Simulated Annealing (SA), Tabu Search, and other iterated local search techniques operate on a single candidate at a time. While SA effectively escapes local minima through probabilistic acceptance of worse moves, these single-solution methods lack population diversity, which is an essential property for maintaining Pareto-optimal fronts in multi-objective domains [37]. Bayesian Optimization methods, including Gaussian Processes, Tree-structured Parzen Estimators (TPE), and Random Forest surrogates, build probabilistic models of the objective function. They perform well for low-dimensional, continuous spaces but scale poorly to high-dimensional, discrete, or noisy plan spaces [38]. Estimation of Distribution Algorithms (EDA) face similar difficulties when evaluation noise dominates signal.

Reinforcement Learning (RL) provides another promising method for sequential decision-making. Techniques such as policy gradients and deep Q-learning have achieved success in dynamic environments [39]. However, they require extensive training data, careful reward shaping, and substantial computational resources. More critically, their black-box nature limits interpretability, which is an essential property for doctrinally grounded military planning and a transparency is a strong requirement for this thesis.

In summary, while many alternative optimization methods exist, the GA was selected as the most pragmatic choice. It balances tractability, robustness to stochastic evaluations, and compatibility with hierarchical task networks, while preserving transparency and interpretability. This aligns with the engineering philosophy of “make it work, make it right, make it fast”: the GA provided a reliable baseline within the runtime and complexity constraints of this project, leaving room for more advanced methods in future work.

5.5 Hyperparameter Optimization

The HPO procedure was intentionally lightweight, balancing exploration depth with available runtime. Each configuration was tested under limited generations and population sizes, which restricted the search but still yielded consistent results across multiple rosters. More extensive sweeps, or joint HPO across rosters, would likely improve generality but at significant computational cost. Parallelization and more efficient evaluation models could mitigate this constraint in future work.

TODO



Chapter 6

Conclusion

This thesis set out to explore whether doctrinally grounded, agent-based coordination for military operations could be generated through evolutionary optimization guided by HTNs. The experiments demonstrated that hyperparameter-tuned GAs, when integrated with HTN decomposition, are effective at producing plans that are both tactically robust and interpretable to human operators.

The research questions guiding this work were framed around adaptability, interpretability, and coordination. The following sections revisit each question in turn, summarizing the main findings and their implications.

RQ1: Dynamic Replanning within Hierarchical Frameworks

The integration of HTNs with GA proved to be working well. HTNs provided a doctrinal backbone that constrained the search space to valid and meaningful plan structures, while the GA introduced adaptability by optimizing sequencing and parameters under stochastic battlefield conditions. This combination ensured that all generated plans respected doctrinal norms while still exploiting flexibility for optimization. The results confirmed that HTN guidance not only preserved validity but also improved search efficiency, enabling the GA to focus exploration on promising regions of the strategy space. The synergy between HTNs and GA therefore stands as a central contribution of this work.

RQ2: Constraints and Heterogeneous Capabilities

Through ABM terrain features, cost limits, and heterogeneous unit roles were successfully incorporated, optimization consistently produced doctrinally plausible solutions: anti-tank groups prioritized armor, while artillery and infantry provided suppression and support. The hyperparameters discovered through HPO generalized across different rosters, producing high-quality solutions in armor-heavy scenarios. Transferability was weaker when armor was absent and pressure came primarily from infantry and artillery, but even then the optimizer adapted by selecting leaner, efficiency-oriented rosters.

A clear trade-off emerged between robustness and efficiency. When time penalties were absent, solutions emphasized durability and achieved near-perfect success rates, albeit over longer engagements. When penalties were introduced, optimization favored faster but more brittle plans that could fail under unfavorable conditions. This robustness–efficiency balance shows how objective formulation directly shapes tactical outcomes and provides an explicit mechanism for encoding command priorities within the optimization process.

RQ3: Human–Machine Collaboration and Usability

The framework emphasized interpretability and usability for human operators. Plans were compact, expressed as sequences of doctrinally valid tasks, and therefore easy to understand. Visualization on the operational grid, supported by Gantt-style charts, provided clear insight into how coordinated actions unfolded over time. Robustness across seeds further strengthened confidence that results reflected stable tactical behavior rather than random variation.

A deliberate design choice was the use of genetic algorithms rather than opaque, black-box learning models such as deep reinforcement learning. This ensured that the optimization process itself remained transparent: each plan could be traced back to interpretable operations such as crossover, mutation, and selection, rather than to opaque network parameters. This transparency allows human operators to understand not only the resulting plans but also how those plans emerged through the evolutionary process.

The ability to contrast robust but slower solutions against faster but riskier alternatives gives commanders an explicit means of aligning algorithmic planning with mission intent, strengthening the framework’s role as a collaborative decision-support tool.

Future Work

Future extensions should broaden the doctrinal template space by incorporating a richer set of tactics, reserves, and logistic considerations. The current framework focuses primarily on the *combat* function, but real operational plans also include extensive provisions for *combat support* (e.g., reconnaissance, engineering, electronic warfare) and *combat service support* (e.g., logistics, medical evacuation, resupply). Expanding the optimizer to include these dimensions would improve realism and doctrinal completeness, though at the cost of a much larger search space. 

Further realism could be introduced by operating on real geographical data instead of procedurally generated terrain. Loading and classifying real maps would enable pathfinding and tactical reasoning to operate directly on actual terrain features such as roads, forests, and built-up areas. Once robust plans are identified, a natural next step would be to validate them within high-fidelity training environments such as *VBS4*¹, bridging the gap between algorithmic generation and professional simulation. Such integration would provide opportunities for human-in-the-loop assessment and operational evaluation. 

Hyperparameter optimization also offers several opportunities for refinement. The present work relied on light HPO sweeps to maintain runtime feasibility, but more comprehensive searches would likely yield better-tuned evolutionary dynamics. A promising next step is to conduct joint HPO across multiple rosters—optimizing for average or worst-case performance—to produce roster-agnostic parameter settings and improve generality across scenarios. Achieving this, however, will require greater computational resources and improved runtime scaling. 

Finally, efficiency remains a central challenge. Even with modest GA budgets (20 population, 10 generations), the optimization process required significant computation time. Following the philosophy of “make it work, make it right, make it fast,” this thesis prioritized feasibility and methodological soundness. Future research should therefore focus on scaling the optimization pipeline and developing interactive interfaces that enable commanders to steer or refine evolutionary search directly. Such human-machine collaboration would ensure that optimization remains not only doctrinally grounded but also operationally usable.

¹<https://bisimulations.com/products/vbs4/>

Closing Statement

In conclusion, this thesis demonstrated that hyperparameter-optimized genetic algorithms, when combined with hierarchical task networks, can generate doctrinally meaningful, interpretable, and effective coordination plans. The framework proved robust across mixed and armor-heavy scenarios, confirming that embedding doctrinal structure within evolutionary optimization can bridge the gap between abstract algorithmic search and operationally relevant battle planning. The integration of GA and HTN thus represents a practical step toward transparent, adaptive, and human-usable military decision-support systems.



Chapter 7

Ethics and Data Management

The ethics of warfare is a heavily debated field, traditionally framed by Just War Theory (bellum iustum). This doctrine sets moral criteria for both the justification of war and its conduct [40]. Jus ad bellum (the right to wage war) requires a just cause, a legitimate authority, proportionality, it must be the last resort, and have a reasonable chance of success. Jus in bello (right conduct in war) demands discrimination between combatants and civilians, proportionality in the use of force, and prohibits torture or deliberate civilian targeting. As Walzer notes in *Just and Unjust Wars* (1977), war may be justified to confront grave wrongs, but only if conducted ethically, cautioning against reducing moral judgment to mere cost-benefit calculation [41].

The use of AI in warfare complicates these principles. Black-box systems lack transparency and accountability, risking errors or biases that violate Just War constraints. For example, an AI targeting algorithm could misclassify a civilian vehicle, breaching discrimination, while an autonomous drone might escalate conflict with disproportionate strikes. These problems are intensified by accountability gaps: when machines are delegated lethal authority, responsibility for harm becomes unclear.

This dissertation's framework mitigates such risks by embedding transparency and oversight into tactical planning. Agent-based modeling (ABM) makes agent behaviors explicit, hierarchical task networks (HTNs) align plans with doctrinal rules, and genetic algorithms generate auditable decision pathways. Together, these methods provide explainability absent in black-box AI. Human oversight at critical points maintains accountability, ensuring compliance with jus in bello by minimizing civilian harm and supporting jus ad bellum through doctrinally grounded strategies. The result is not only a technically viable system but also an ethically robust alternative for AI-supported military planning.

I acknowledge that the thesis adheres to the ethical code (<https://student.uva.nl/en/topics/ethics-in-research>) and research data management policies (<https://rdm.uva.nl/en>) of UvA and IvI.

The following table lists the data used in this thesis (including source codes). I confirm that the list is complete and the listed data are sufficient to reproduce the results of the thesis. If a prohibitive non-disclosure agreement is in effect at the time of submission "NDA" is written under "Availability" and "License" for the concerned data items.

Short description (max. 10 words)	Availability (e.g., URL, DOI)	License (e.g., MIT, GPL)
Example dataset 1	<github url>or Figshare	GPL
Example source code	DOI (from Zenodo)	MIT
Example sensitive data	NDA	NDA

TODO

Appendix A

State Templates

```
TANK_GROUP_SIZE = 12
INFANTRY_GROUP_SIZE = 12
ARTILLERY_GROUP_SIZE = 3
ANTI_TANK_GROUP_SIZE = 2

ENEMY_TANK_GROUP_SIZE = 4
ENEMY_INFANTRY_GROUP_SIZE = 4
ENEMY_ANTI_TANK_GROUP_SIZE = 2
ENEMY_ARTILLERY_GROUP_SIZE = 1

tank_state_template = {
    "type": "tank",
    "position": (0, GRID_HEIGHT),
    "facing": (0, 1),
    "speed": (((75 / 3.6) * 6) / CELL_SIZE),
    "move_credit": 0,
    "base_health": 20,
    "health": 20 * TANK_GROUP_SIZE,
    "max_health": 20 * TANK_GROUP_SIZE,
    "group_size": TANK_GROUP_SIZE,
    "current_group_size": TANK_GROUP_SIZE,
    "cumulative_damage": 0.0,
    "armor_front": 17,
    "armor_side": 4,
    "armor_rear": 3,
    "friendly_accuracy": 0.75,
```

```
"base_rate_of_fire": 4.9,
"damage": 9,
"suppression": 0.12,
"penetration": 18,
"vision_range": 2000 / CELL_SIZE,
"attack_range": 1200 / CELL_SIZE,
"all_enemies_spotted": False,
"suppression_from_enemy": 0.0,
"stealth_modifier": 0
}

infantry_state_template = {
    "type": "infantry",
    "position": (1, GRID_HEIGHT),
    "facing": (0, 1),
    "speed": (((65 / 3.6) * 6) / CELL_SIZE),
    "move_credit": 0,
    "base_health": 1,
    "health": 1 * INFANTRY_GROUP_SIZE,
    "max_health": 1 * INFANTRY_GROUP_SIZE,
    "group_size": INFANTRY_GROUP_SIZE,
    "current_group_size": INFANTRY_GROUP_SIZE,
    "cumulative_damage": 0.0,
    "armor_front": 0,
    "armor_side": 0,
    "armor_rear": 0,
    "friendly_accuracy": 0.50,
    "base_rate_of_fire": 294,
    "damage": 0.8,
    "suppression": 0.01,
    "penetration": 1,
    "vision_range": 2200 / CELL_SIZE,
    "attack_range": 600 / CELL_SIZE,
    "all_enemies_spotted": False,
    "suppression_from_enemy": 0.0,
    "stealth_modifier": 0
}

artillery_state_template = {
```

```

    "type": "artillery",
    "facing": (0, 1),
    "speed": (((65 / 3.6) * 6) / CELL_SIZE),
    "move_credit": 0,
    "base_health": 18,
    "health": 18 * ARTILLERY_GROUP_SIZE,
    "max_health": 18 * ARTILLERY_GROUP_SIZE,
    "group_size": ARTILLERY_GROUP_SIZE,
    "current_group_size": ARTILLERY_GROUP_SIZE,
    "cumulative_damage": 0.0,
    "armor_front": 2,
    "armor_side": 1,
    "armor_rear": 1,
    "friendly_accuracy": 0.85,
    "base_rate_of_fire": 8.6,
    "damage": 3.5,
    "suppression": 15,
    "penetration": 1,
    "vision_range": 1800 / CELL_SIZE,
    "attack_range": 2000 / CELL_SIZE,
    "all_enemies_spotted": False,
    "suppression_from_enemy": 0.0,
    "stealth_modifier": 0
}

```

```

anti_tank_state_template = {
    "type": "anti-tank",
    "position": (3, GRID_HEIGHT),
    "facing": (0, 1),
    "speed": (((65 / 3.6) * 6) / CELL_SIZE),
    "move_credit": 0,
    "base_health": 18,
    "health": 18 * ANTI_TANK_GROUP_SIZE,
    "max_health": 18 * ANTI_TANK_GROUP_SIZE,
    "group_size": ANTI_TANK_GROUP_SIZE,
    "current_group_size": ANTI_TANK_GROUP_SIZE,
    "cumulative_damage": 0.0,
    "armor_front": 2,
    "armor_side": 1,

```



```
"armor_rear": 1,
"friendly_accuracy": 0.90,
"base_rate_of_fire": 6.3,
"damage": 15,
"suppression": 0.10,
"penetration": 22,
"vision_range": 2000 / CELL_SIZE,
"attack_range": 1400 / CELL_SIZE,
"all_enemies_spotted": False,
"total_enemies": 2,
"suppression_from_enemy": 0.0,
"scout_steps": 0,
"stealth_modifier": 50
}

enemy_tank_state_template = {
    "type": "enemy_tank",
    "position": (GRID_WIDTH, 1),
    "facing": (0, 1),
    "speed": ((85 / 3.6) * 6) / CELL_SIZE,
    "move_credit": 0,
    "enemy_alive": True,
    "health": 25 * ENEMY_TANK_GROUP_SIZE,
    "max_health": 25 * ENEMY_TANK_GROUP_SIZE,
    "base_health": 25,
    "group_size": ENEMY_TANK_GROUP_SIZE,
    "current_group_size": ENEMY_TANK_GROUP_SIZE,
    "cumulative_damage": 0.0,
    "armor_front": 18,
    "armor_side": 5,
    "armor_rear": 3,
    "outpost_position": (GRID_WIDTH - 1, 0),
    "attack_range": 1200 / CELL_SIZE,
    "accuracy": 0.7,
    "penetration": 18,
    "damage": 9,
    "suppression": 0.12,
    "base_rate_of_fire": 5.5,
    "suppression_from_enemy": 0.0,
```

```

    "vision_range": 1800 / CELL_SIZE,
    "retreat_point": (GRID_WIDTH - 1, GRID_HEIGHT - 1),
    "stealth_modifier": 0
}

```

```

enemy_infantry_state_template = {
    "type": "infantry",
    "position": (GRID_WIDTH - 1, 1),
    "facing": (0, 1),
    "speed": (((75 / 3.6) * 6) / CELL_SIZE),
    "move_credit": 0,
    "enemy_alive": True,
    "base_health": 1,
    "health": 1 * ENEMY_INFANTRY_GROUP_SIZE,
    "max_health": 1 * ENEMY_INFANTRY_GROUP_SIZE,
    "group_size": ENEMY_INFANTRY_GROUP_SIZE,
    "current_group_size": ENEMY_INFANTRY_GROUP_SIZE,
    "cumulative_damage": 0.0,
    "armor_front": 0,
    "armor_side": 0,
    "armor_rear": 0,
    "outpost_position": (GRID_WIDTH - 1, 0),
    "accuracy": 0.50,
    "base_rate_of_fire": 294,
    "damage": 0.8,
    "suppression": 0.01,
    "penetration": 1,
    "attack_range": 600 / CELL_SIZE,
    "all_enemies_spotted": False,
    "suppression_from_enemy": 0.0,
    "current_patrol_index": 0,
    "vision_range": 2000 / CELL_SIZE,
    "stealth_modifier": 0
}

```

```

enemy_anti_tank_state_template = {
    "type": "anti-tank",
    "position": (GRID_WIDTH - 2, 1),
    "facing": (0, 1),
    "speed": (((75 / 3.6) * 6) / CELL_SIZE),

```

```

    "move_credit": 0,
    "enemy_alive": True,
    "base_health": 18,
    "health": 18 * ENEMY_ANTI_TANK_GROUP_SIZE,
    "max_health": 18 * ENEMY_ANTI_TANK_GROUP_SIZE,
    "group_size": ENEMY_ANTI_TANK_GROUP_SIZE,
    "current_group_size": ENEMY_ANTI_TANK_GROUP_SIZE,
    "cumulative_damage": 0.0,
    "armor_front": 2,
    "armor_side": 1,
    "armor_rear": 1,
    "outpost_position": (GRID_WIDTH - 1, 0),
    "accuracy": 0.90,
    "base_rate_of_fire": 6.3,
    "damage": 15,
    "suppression": 0.1,
    "penetration": 22,
    "attack_range": 1400 / CELL_SIZE,
    "all_enemies_spotted": False,
    "suppression_from_enemy": 0.0,
    "vision_range": 2000 / CELL_SIZE,
    "stealth_modifier": 50
}

```

```

enemy_artillery_state_template = {
    "type": "artillery",
    "position": (GRID_WIDTH - 3, 1),
    "facing": (0, 1),
    "speed": (((75 / 3.6) * 6) / CELL_SIZE),
    "move_credit": 0,
    "enemy_alive": True,
    "base_health": 13,
    "health": 13 * ENEMY_ANTI_TANK_GROUP_SIZE,
    "max_health": 13 * ENEMY_ANTI_TANK_GROUP_SIZE,
    "group_size": ENEMY_ANTI_TANK_GROUP_SIZE,
    "current_group_size": ENEMY_ANTI_TANK_GROUP_SIZE,
    "cumulative_damage": 0.0,
    "armor_front": 2,
    "armor_side": 1,

```

```
    "armor_rear": 1,  
    "outpost_position": (GRID_WIDTH - 1, 0),  
    "accuracy": 0.90,  
    "base_rate_of_fire": 8.6,  
    "damage": 3.5,  
    "suppression": 0.15,  
    "penetration": 0,  
    "attack_range": 2200 / CELL_SIZE,  
    "all_enemies_spotted": False,  
    "suppression_from_enemy": 0.0,  
    "vision_range": 1800 / CELL_SIZE,  
    "stealth_modifier": 0  
}
```

Appendix B

Child Mutation Operations

In the genetic algorithm described in Section 3.7.4, mutation serves as the primary source of structural variation once crossover has combined information from two parents. The following figures illustrate the three types of mutation operators applied to child chromosomes. Each operator modifies the attack sequence or team assignments in a distinct way, ensuring continued exploration of the plan space while preserving doctrinal constraints.

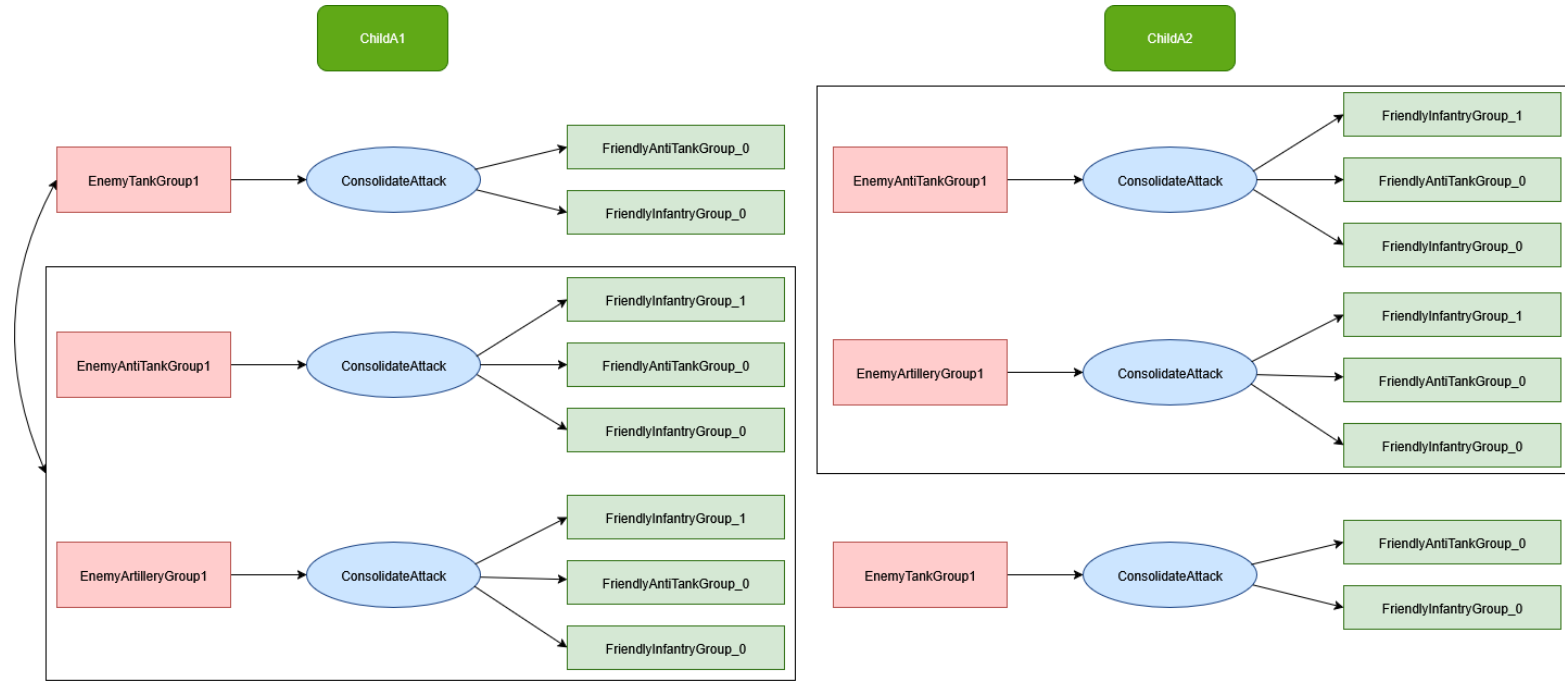


FIGURE B.1: Mutation operator (a): Shifting a block to different location in the sequence.

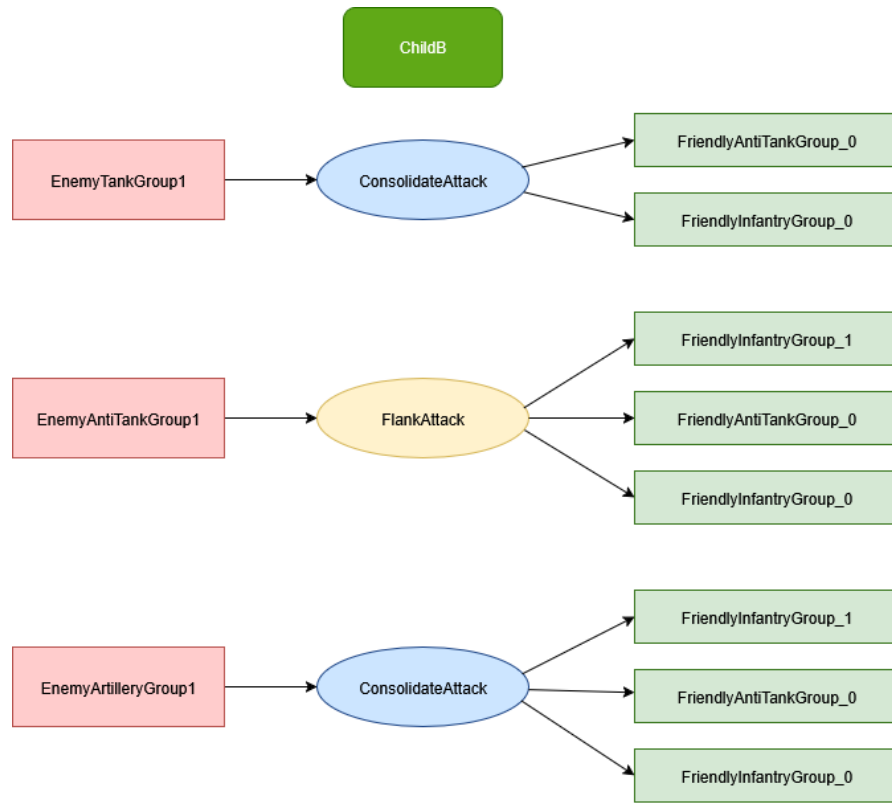


FIGURE B.2: Mutation operator (b): Flipping of attack type.

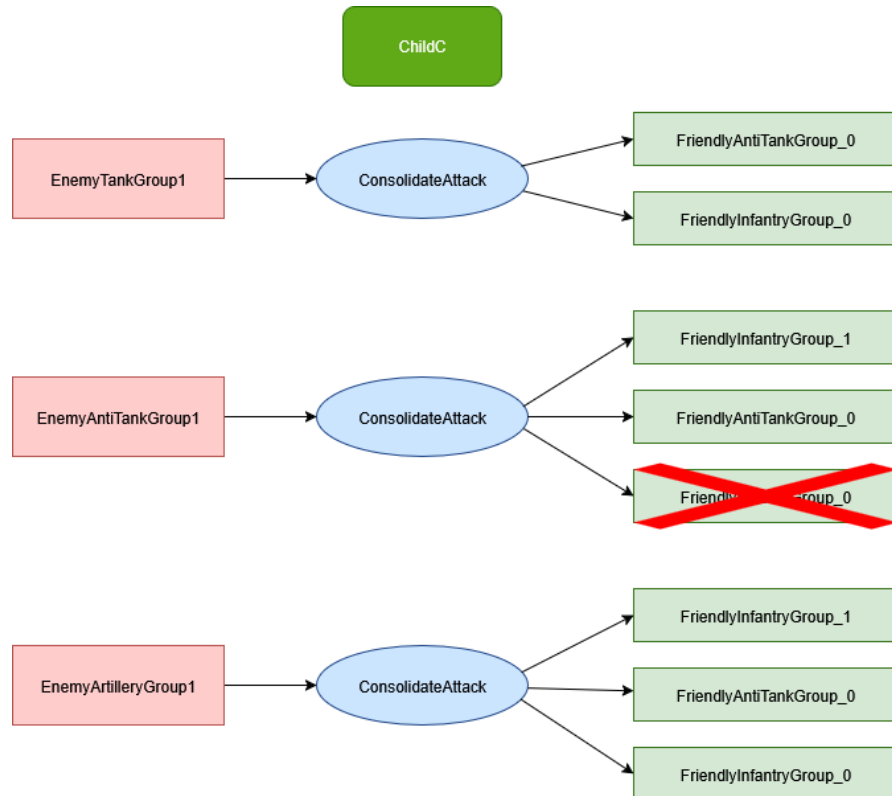


FIGURE B.3: Mutation operator (c): Adding or removing a friendly unit.

Bibliography

- [1] Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning: theory and practice*. Elsevier, 2004.
- [2] *IK 2:17, 10th impression: Troepenaanvoering (for the tactical level)*. Royal Netherlands Army, Land Warfare Department, n.d. In Dutch.
- [3] Thomas Douglas Sloan. Repetitive use of joint theater level simulation (jtls) for investigation of headquarters effectiveness. 1988.
- [4] Dod instruction 5000.61: Dod modeling and simulation (m&s) verification, validation, and accreditation. Technical Report 5000.61, Department of Defense, May 2003. Issued on May 13, 2003. Also cited as DoDI 6000.61.
- [5] Bernard P Zeigler, Herbert Praehofer, and Tag Gon Kim. *Theory of modeling and simulation*. Academic press, 2000.
- [6] Mark A. Bedau. Weak emergence and computer simulation. In Paul Humphreys and Cyrille Imbert, editors, *Models, Simulations, and Representations*, pages 91–114. Routledge, New York, 2011.
- [7] Eric Winsberg. Computer simulations in science. 2013.
- [8] Joshua M Epstein. Generative social science: Studies in agent-based computational modeling. In *Generative Social Science*. Princeton University Press, 2012.
- [9] Stuart J Russell and Peter Norvig. *Artificial intelligence: a modern approach*. pearson, 2016.
- [10] Che Wang, Pan Chen, Yuanda Li, Christoffer Holmgård, and Julian Togelius. Portfolio online evolution in starcraft. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 12, pages 114–120, 2016.
- [11] Kristina Lerman, Chris Jones, Aram Galstyan, and Maja J. Matarić. Analysis of dynamic task allocation in multi-robot systems. *The International Journal of Robotics Research*, 25(4):225–241, 2006.

- [12] Byron R Harder. *Automated battle planning for combat models with maneuver and fire support*. PhD thesis, Monterey, California: Naval Postgraduate School, 2017.
- [13] Joshua M. Epstein. Modeling civil violence: An agent-based computational approach. *Proceedings of the National Academy of Sciences*, 99(suppl 3):7243–7250, 2002. doi: 10.1073/pnas.092080199. URL http://www.pnas.org/content/99/suppl_3/7243.abstract.
- [14] Jo Erskine Hannay, Ole Martin Mevassvik, Arild Skjeltorp, and Karsten Brathen. Lvc simulation for land operations training. *Norwegian Defence Research Establishment (FFI), PO Box*, 25.
- [15] Svajone BEKESIENE and Gediminas Prūsevičius. Simulators usage assessment for higher military readiness. *CHALLENGES TO NATIONAL DEFENCE IN CONTEMPORARY GEOPOLITICAL SITUATION CNDCGS2020*, page 114, 2020.
- [16] Andy Ilachinski. Irreducible semi-autonomous adaptive combat (isaac): An artificial-life approach to land combat. *Military Operations Research*, pages 29–46, 2000.
- [17] Nazli N El Samaloty, Roger Schleper, Mary Anne Fawkes, and Dean Muscietta. Infantry warrior simulation (iwars): a soldier-centric constructive simulation. *Phalanx*, 40(2):29–31, 2007.
- [18] Imre Balogh, Curtis L Blais, David Reeves, and Kirk Stork. Modeling expeditionary advanced base operations in the combined arms analysis tool for the 21st century (combatxxi). 2022.
- [19] K. Erol, D. S. Nau, and J. Hendler. Htn planning: Complexity and expressivity. In *AAAI Fall Symposium on Planning and Learning*, pages 112–119. AAAI, 1994.
- [20] Richard E Fikes and Nils J Nilsson. Strips: A new approach to the application of theorem proving to problem solving. *Artificial intelligence*, 2(3-4):189–208, 1971.
- [21] Richard E Fikes, Peter E Hart, and Nils J Nilsson. Learning and executing generalized robot plans. *Artificial intelligence*, 3:251–288, 1972.
- [22] Hai Hoang, Stephen Lee-Urban, and Héctor Muñoz-Avila. Hierarchical plan representations for encoding strategic game ai. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 1, pages 63–68, 2005.
- [23] Peter Gorniak and Ian Davis. Squadsmart—hierarchical planning and coordinated plan execution for squads of characters. In *Proceedings of the AAAI Conference*

- on *Artificial Intelligence and Interactive Digital Entertainment*, volume 3, pages 14–19, 2007.
- [24] John-Paul Kelly, Adi Botea, and Sven Koenig. Offline planning with hierarchical task networks in video games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 4, pages 60–65, 2008.
- [25] Hector Muñoz-Avila and David Aha. On the role of explanation for hierarchical case-based planning in real-time strategy games. In *Proceedings of ECCBR-04 Workshop on Explanations in CBR*, pages 1–10. Citeseer, 2004.
- [26] Santiago Ontanón, Kinshuk Mishra, Neha Sugandh, and Ashwin Ram. Case-based planning and execution for real-time strategy games. In *Case-Based Reasoning Research and Development: 7th International Conference on Case-Based Reasoning, ICCBR 2007 Belfast, Northern Ireland, UK, August 13-16, 2007 Proceedings 7*, pages 164–178. Springer, 2007.
- [27] Santiago Ontanón and Michael Buro. Adversarial hierarchical-task network planning for complex real-time games. In *Proceedings of the 24th International Conference on Artificial Intelligence*, pages 1652–1658, 2015.
- [28] Athanasios Papoulis. *Bernoulli Trials*, pages 57–63. McGraw-Hill, New York, 2 edition, 1984.
- [29] Pratibha Sharma, Manoj Diwakar, and Niranjana Lal. Edge detection using moore neighborhood. *International Journal of Computer Applications*, 61(3), 2013.
- [30] Martin Gardner. *Taxicab Geometry*, chapter 10, pages 159–176. Copernicus, 1997. ISBN 0-387-94929-1.
- [31] J. E. Bresenham. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1):25–30, 1965. doi: 10.1147/sj.41.0025. Archived from the original (PDF) on 2008-05-28.
- [32] Bird’s Eye Games. Regiments. <https://store.steampowered.com/app/1109680/Regiments/>, 2022. Accessed: 2025-10-10.
- [33] Thomas Bartz-Beielstein, Jürgen Branke, Jörn Mehnen, and Olaf Mersmann. Evolutionary algorithms. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 4(3):178–195, 2014.
- [34] A Kai Qin, Vicky Ling Huang, and Ponnuthurai N Suganthan. Differential evolution algorithm with strategy adaptation for global numerical optimization. *IEEE transactions on Evolutionary Computation*, 13(2):398–417, 2008.

- [35] Pablo Moscato, Carlos Cotta, and Alexandre Mendes. Memetic algorithms. In *New optimization techniques in engineering*, pages 53–85. Springer, 2004.
- [36] V Selvi and R Umarani. Comparative analysis of ant colony and particle swarm optimization techniques. *International Journal of Computer Applications*, 5(4):1–6, 2010.
- [37] Habib Youssef, Sadiq M Sait, and Hakim Adiche. Evolutionary algorithms, simulated annealing and tabu search: a comparative study. *Engineering Applications of Artificial Intelligence*, 14(2):167–181, 2001.
- [38] Peter I Frazier. A tutorial on bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.
- [39] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. Deep reinforcement learning: A brief survey. *IEEE signal processing magazine*, 34(6):26–38, 2017.
- [40] Mansi Rathour. Just war to just peace: Jus post bellum for a lasting peace. Master’s thesis, University of Groningen, Groningen, Netherlands, 2018. Supervisor: Dr. E.R. Boot, Institute of Philosophy.
- [41] Michael Walzer. *Just and Unjust Wars*. Basic Books, New York, 4th edition, 2006. ISBN 978-0-465-03707-0.